



华南理工大学  
South China University of Technology

# 专业学位硕士学位论文

文件在线预览服务快速响应技术研究

---

|        |                     |
|--------|---------------------|
| 作者姓名   | 李华青                 |
| 学位类别   | 工程硕士                |
| 指导教师   | 奚建清 教授<br>李慧琪 高级工程师 |
| 所在学院   | 软件学院                |
| 论文提交日期 | 2019 年 3 月          |

# **Research on File Online Preview Service Quick Response Technology**

A Dissertation Submitted for the Degree of Master

**Candidate: Li Huaqing**

**Supervisor: Prof. Xi Jianqing**

South China University of Technology

Guangzhou, China

分类号：TP3

学校代号：10561

学 号：201621035154

## 华南理工大学硕士学位论文

# 文件在线预览服务快速响应技术研究

作者姓名：李华青

指导教师姓名、职称：奚建清 教授；李慧琪 高级工程师

申请学位级别：硕士

学科专业名称：软件工程

研究方向：应用开发技术

论文提交日期： 年 月 日

论文答辩日期：2019 年 3 月 14 日

学位授予单位：华南理工大学

学位授予日期： 年 月 日

答辩委员会成员：

主席： 刘琼

委员： 奚建清 刘艳霞 刘飞 潘勇

# 华南理工大学

## 学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

作者签名：李华青

日期：2019年3月19日

## 学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权单位属华南理工大学。学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许学位论文被查阅（除在保密期内的保密论文外）；学校可以公布学位论文的全部或部分内容，可以允许采用影印、缩印或其它复制手段保存、汇编学位论文。本人电子文档的内容和纸质论文的内容相一致。

本学位论文属于：

☐ 保密（校保密委员会审定涉密学位论文时间：\_\_\_\_年\_\_月\_\_日），于\_\_\_\_年\_\_月\_\_日解密后适用本授权书。

☒ 不保密，同意在校园网上发布，供校内师生和与学校有共享协议的单位浏览；同意将本人学位论文编入有关数据库进行检索，传播学位论文的全部或部分内容。

（请在以上相应方框内打“√”）

作者签名：李华青

日期：2019-3-20

指导教师签名：



日期 2019-3-20

## 摘要

在互联网时代的背景下,查看文件的需求日益增多,将文件下载到本地,然后再打开查看的传统方式,已经不能满足办公节奏日益加快用户的需求。由此,文件在线预览服务得到了广泛的发展和应用,在面临高负载网络请求下如何高效转换和存储文件,减少响应时间是本文研究的重点。

本文主要以 PDF 格式在线预览文件,以各个工具模块为基础,构成了完整的文件在线预览服务。本文主要研究了 PDF 文件解析、基于“热数据”均衡存储对 MongoDB 数据均衡策略进行改进和通过对 Nginx 源码的修改,使得默认的加权轮询算法更新节点服务器的权重。本文主要做了如下几个方面的工作:

一、对现有的文件在线预览服务做了调查,由于 Flash 存在占用资源过高、安全性和可靠性低的问题,本文改变了过去依赖 Flash 进行文件预览的方式,将多种格式的源文件转换为 PDF 文件格式和 HTML 文件格式在线预览。

二、由于一些 PDF 文件过大,加上网络带宽的限制,用户对文件某一章节感兴趣时需要预览整个文件,使得用户等待时间过长,影响用户体验。所以本文研究了 PDF 文件解析,将 PDF 文件按章节切分、存储和传输,减少用户等待的时间。

三、面对高并发的文件转换和切分请求,单个服务器处理能力受限。本文采用集群的方式来处理请求,通过修改 Nginx 的源代码,将 Nginx 默认的加权轮询算法(weighted round robin)改为节点服务器权重实时变化的动态加权轮询算法(dynamic weighted round robin)。

四、改进了 MongoDB 的数据均衡策略,对 MongoDB 集群中的“热数据”均衡问题做了深入研究,提出了基于队列的“热数据”识别算法和数据块(Chunk)迁移策略,实现了“热数据”在节点服务器的均衡分布,提升了 MongoDB 集群的性能。

最后,本文分模块开发了文件在线预览服务的相关工具,降低了系统的耦合性和提高了系统的扩展性,并在高负载网络请求下对文件在线预览服务做了性能测试,验证文件在线预览服务是否实现了预期的功能。

**关键词:** 文件在线预览服务; 快速响应; PDF 文件; MongoDB

# Abstract

In the context of the Internet era, the demand for viewing files is increasing. The traditional way of downloading files to the local and then opening the view is no longer sufficient to meet the needs of users. Therefore, the file online preview service has been widely developed and applied. How to efficiently convert and store files and reduce response time in the face of large load network requests is the focus of this paper.

This article mainly previews files in PDF format and builds a complete online preview service based on each tool module. This paper mainly studies the PDF file parsing, the improvement of MongoDB data equalization strategy based on hotspot data balanced storage and the modification of Nginx source code, so that the default weighted round robin updates the weight of the node server in real time. This article mainly does the following work:

First, This paper investigates the existing online preview service of files. Because Flash has the problem of high resource consumption, low security and low reliability, this article changes the way of relying on Flash for file preview in the past, and converts source files of multiple formats into Online preview of PDF file format and HTML file format.

Second, because some PDF files are too large, and the network bandwidth is limited, users need to preview the entire file when they are interested in a certain chapter of the file, causing the user to wait too long and affect the user experience. Therefore, this paper studies the PDF file parsing, segmenting, storing and transmitting PDF files according to chapters, reducing the waiting time of users.

Third, in the face of high concurrency file conversion and split request, a single server processing capacity is limited. This paper uses the cluster method to process the request. By modifying the source code of Nginx, the Nginx default weighted round robin algorithm is changed to the dynamic weighted round robin of the node server weight real-time change.

Fourthly, the data balancing strategy of MongoDB is improved. The hot data equalization problem in MongoDB cluster is deeply studied. The queue-based hot data identification algorithm and the data chunk migration strategy are proposed to realize the hot data in the node server. Distributed balance improves the performance of MongoDB clusters.

Finally, this paper develops tools related to the online preview service of the file, which

reduces the coupling of the system and improves the scalability of the system. The performance of the online preview service of the file is tested under the request of a heavy load network. Whether the online preview service implements the expected functionality.

**Keywords:** File online preview service; Quick response; PDF file; MongoDB

# 目 录

|                              |    |
|------------------------------|----|
| 摘 要 .....                    | I  |
| Abstract .....               | II |
| 第一章 绪论 .....                 | 1  |
| 1.1 研究背景和意义 .....            | 1  |
| 1.2 国内外研究现状 .....            | 2  |
| 1.2.1 MongoDB 数据均衡研究现状 ..... | 3  |
| 1.2.2 PDF 文件转换研究现状 .....     | 3  |
| 1.2.3 负载均衡研究现状 .....         | 4  |
| 1.3 本文的主要工作和创新点 .....        | 5  |
| 1.4 论文组织结构 .....             | 6  |
| 1.5 本章小结 .....               | 6  |
| 第二章 相关基础知识介绍 .....           | 7  |
| 2.1 PDF 文件 .....             | 7  |
| 2.1.1 PDF 文件逻辑结构 .....       | 7  |
| 2.1.2 PDF 文件内容解析 .....       | 8  |
| 2.2 MongoDB 数据库 .....        | 8  |
| 2.2.1 MongoDB 数据库特点 .....    | 8  |
| 2.2.2 MongoDB 自动分片机制 .....   | 9  |
| 2.3 缓存技术 .....               | 11 |
| 2.4 反向代理技术 .....             | 11 |
| 2.5 本章小结 .....               | 11 |
| 第三章 文件在线预览服务需求分析与设计 .....    | 12 |
| 3.1 文件在线预览服务需求分析 .....       | 12 |
| 3.1.1 功能需求 .....             | 12 |
| 3.1.2 性能需求 .....             | 14 |
| 3.2 文件在线预览服务设计 .....         | 15 |
| 3.2.1 功能结构设计 .....           | 15 |



|                               |    |
|-------------------------------|----|
| 3.2.2 性能优化设计 .....            | 20 |
| 3.3 本章小结 .....                | 24 |
| 第四章 基于 MongoDB 数据均衡策略研究 ..... | 25 |
| 4.1 MongoDB 数据均衡策略及问题分析 ..... | 25 |
| 4.1.1 数据块(chunk)的拆分 .....     | 25 |
| 4.1.2 数据均衡策略 .....            | 26 |
| 4.1.3 数据均衡策略不足 .....          | 27 |
| 4.2 改进的数据均衡策略 .....           | 27 |
| 4.2.1 基于队列的热数据识别算法 .....      | 28 |
| 4.2.2 数据块热权重的计算 .....         | 29 |
| 4.2.3 数据块迁移策略 .....           | 30 |
| 4.3 本章小结 .....                | 32 |
| 第五章 文件在线预览服务的实现 .....         | 33 |
| 5.1 开发环境 .....                | 33 |
| 5.2 文件在线预览服务的开发和实现 .....      | 33 |
| 5.2.1 文件上传工具 .....            | 33 |
| 5.2.2 文件预览工具 .....            | 37 |
| 5.2.3 文件下载工具 .....            | 42 |
| 5.2.4 文件转换和切分工具 .....         | 44 |
| 5.3 工具展示 .....                | 49 |
| 5.4 本章小结 .....                | 50 |
| 第六章 文件在线预览服务实验分析 .....        | 51 |
| 6.1 实验环境 .....                | 51 |
| 6.1.1 网络拓扑结构 .....            | 51 |
| 6.1.2 实验环境硬件配置 .....          | 52 |
| 6.1.3 实验环境软件配置 .....          | 52 |
| 6.2 功能测试 .....                | 53 |
| 6.3 集群性能测试 .....              | 54 |
| 6.3.1 方案 1 测试和分析 .....        | 55 |
| 6.3.2 方案 2 测试和分析 .....        | 58 |

|                       |    |
|-----------------------|----|
| 6.4 本章小结 .....        | 60 |
| 总结与展望 .....           | 61 |
| 论文总结 .....            | 61 |
| 下一步工作 .....           | 61 |
| 参考文献 .....            | 62 |
| 攻读硕士学位期间取得的研究成果 ..... | 65 |
| 致谢 .....              | 66 |

# 第一章 绪论

## 1.1 研究背景和意义

当今时代是一个信息化的时代，网络服务已经渗透到人们生活的各个角落。尤其是近些年，在政府大力推动互联网快速发展的时代背景下，无论是企业还是政府单位，为了提高办公的效率，很多业务都以在线服务的形式展开。由于电子文件存储、传输方便快捷，所以在文件数量呈现爆炸式增长的环境下，大量的文件资料、数据都以电子文件的形式存储和记录。如何快速、高效的查看这些电子文件显得尤为重要。

在文件查看的实际使用过程中，用户需要先将文件下载到本地，然后使用本地的软件查看。而这些文件大小不一，往往从几 MB 到几十 MB，甚至是几百 MB，再加上网络带宽的限制，下载文件到本地的时间也很可观<sup>[1]</sup>。如果用户本地没有查看文件相应的软件，用户还得下载专业的软件，有些软件不仅安装麻烦，而且收费，这些过程占用了用户大量的时间和精力，不能达到用户及时查看想要查阅文件的要求，给用户带来了极其不好的办公体验。因此，这种传统的文件阅读模式已经不能满足办公节奏日益加快的用户需求，文件在线预览服务得到了广泛的发展和应用。

文件在线预览服务可以不用下载文件，直接在浏览器端在线预览文件，减少用户的等待时间。该服务有效的解决了因文件格式不同而需要不同本地软件打开的问题，统一在浏览器上预览文件；为了减少用户的等待时间，该服务将文件分章节切分、存储，用户需要预览某一章节，传输此章节，将网络带宽限制的影响降到最低。因此，本文件在线预览服务实现了不同文件格式的统一预览和定点章节的预览，提高了用户的办公效率。

由于 Flash 存在占用资源过高、安全性和可靠性低的问题，而且 Adobe 即将在 2020 年停止和分发 Flash<sup>[2]</sup>。所以本文抛弃以往文件预览系统的做法，不再使用 Flash 进行文件预览。而且现在 PDF 文件的广泛使用和各浏览器都在线支持 PDF 格式浏览，所以本文件在线预览服务将用户上传的文件转换为 PDF 格式的文件进行预览。由于移动端的普及带来的大量互联网用户，但是移动端的有些浏览器不支持 PDF 文件的预览，为了提高用户体验，本在线服务将 PDF 格式转换为 HTML 格式在移动端进行预览。

使用文件在线预览服务的用户往往更加关注响应时间，这就要求服务有很高的性能完成用户的预览请求。在提升服务的速度和性能的研究方向中，主要有：一方面研究处理速度更快的计算机，另一方面是研究分布式系统。但是随着计算机处理速度的提升，其价格也在飞速上升，高昂的价格使得普通的企业难以承受。所以对于大多数企业而言，

对系统进行横向扩展是更好的选择，利用价格低廉的服务器组成集群系统，使得系统的性能提升和获得更低的响应时间<sup>[3]</sup>。

由多台服务器组成的集群，必须保证用户的请求能够合理的分配在各个节点服务器上，这样才能提升系统的整体性能。不然，由于存在“木桶效应”使得负载过大的节点服务器成为整个系统的短板，而且负载很低节点服务器的资源没有得到充分使用，使得整个在线服务的响应时间过大<sup>[4]</sup>。在本文件在线预览服务中，如何分配用户请求主要体现在两个方面：一方面是在大量用户上传文件时，系统面临高并发的文件转换和切分请求，需要使用负载均衡策略将文件转换和切分任务平均分配到各个节点服务器上；另一方面是在现实的存储环境中数据的访问呈现相当高的局部性，根据访问频率的不同，频繁访问的数据为“热数据”，很少或者从不访问的数据为“冷数据”，所以需要将“冷热数据”均匀存储在节点服务器中，使得节点服务器能够高效的处理用户请求。因此在高负载网络请求中，如何有效的处理用户的请求是本文研究的重点。

文件在线预览服务涉及到文件的上传、下载、预览、转换和存储，如何对此服务进行解耦，提高开发效率也是本文亟需解决的问题。

本文是实验室的科研项目“智慧科技平台及其关键技术研究”的一部分，主要解决文件在线预览服务中遇到的文件转换和切分问题、高负载网络请求高性能处理问题和文件存储问题。为了降低此服务的开发难度，本人使用了实验室科研团队提出的“工具模型”，对此服务进行解耦，提高服务的扩展性和工具的重用性。

本文的研究工作主要有：文件在线预览服务相关工具的开发、PDF 文件结构的研究、对 Nginx 默认的加权轮询算法进行改进和基于 MongoDB 的冷热数据均衡分布。服务相关工具的开发对文件在线预览服务进行解耦，降低开发难度；PDF 文件结构研究能够提取 PDF 文件大纲，按目录切分文件；将 Nginx 默认的加权轮询算法改为权重的动态更新，更好的对文件转换和切分工具进行负载均衡；MongoDB 冷热数据均衡让数据按访问热度进行均衡存储，减小节点服务器的访问压力。最终开发的文件在线预览服务成功上线运行，达到了预期的效果。

## 1.2 国内外研究现状

本文主要通过 PDF 文件按章节切分，减少数据量和采用集群的方式，减少 HTTP 请求、SQL 请求的排队时间来减少系统的响应时间，快速响应用户的请求，为此研究了如下三个方面国内外的研究现状。

### 1.2.1 MongoDB 数据均衡研究现状

MongoDB 作为新一代的 NoSQL 数据库,与传统的数据库相比较,在获取数据上不仅更加便捷高效,而且还拥有 SQL 的索引功能,同时还拥有自动分片的特点。<sup>[5-7]</sup>

MongoDB 的自动分片(Auto-Sharding)技术是在存储集群中将数据分片存储在不同的节点服务器中,为了减轻节点服务器的读写压力,减少系统的响应时间,就必须对分片中的数据块(Chunk)进行迁移,使得各个分片之间数据均衡。虽然 MongoDB 自带的方法能够均衡数据,但是没有考虑在实际的情况下数据有着冷热之分,虽然各个节点服务器之间的数据量均衡,但是节点服务器也会因冷热数据分布不均,使得有的节点服务器访问压力大,有的访问压力小,从而降低整个集群的性能。

针对这一问题,国内外的学者也有相应的研究。文献<sup>[8]</sup>首先分析了 NoSQL 数据库的数据均衡策略,然后介绍了 MongoDB 数据块(Chunk)迁移的缺陷:在数据插入操作过快的时候会使得各个节点服务器频繁交换数据,降低服务器的性能。为此,作者提出了基于 Chord 环的数据均衡策略代替 MongoDB 原有的策略;文献<sup>[7]</sup>在分析 MongoDB 存储数据的实际访问特点上,对影响集群性能操作做了评估实验,针对 CRUD 操作提出了 FODO 算法,该算法通过评估节点服务器的性能,进行 Chunk 迁移,改善了 MongoDB 的分片机制,提高了 MongoDB 的读写性能;文献<sup>[9]</sup>通过研究 MongoDB 的分片机制和数据均衡策略,提出了基于 Markov 的数据均衡策略;文献<sup>[10]</sup>分析了 MongoDB 的数据均衡机制,提出了优化的 MongoDB 数据均衡算法,其核心思想为“预防为主,最低消耗处理”在此基础之上实现了基于访问的负载均衡,达到了降低资源消耗的目标。

本文为了提高文件在线预览服务的性能,改进了 MongoDB 的存储性能,对 MongoDB 集群中的冷热数据均衡问题做了深入研究,提出了基于队列的热数据识别算法、数据块(Chunk)热权重计算和迁移策略,实现了冷热数据在节点服务器的均衡,提升了 MongoDB 集群的性能,达到了预期的效果。

### 1.2.2 PDF 文件转换研究现状

关于 PDF 文件的解析,国内外的研究主要集中在如何将 PDF 文件转换为其它格式的文件、PDF 文件表格识别等,也有一些学者研究 PDF 文件图像和图形的处理、PDF 内容组织和内容标注等。但是对 PDF 文件内容解析的相关研究比较少。

文献<sup>[11]</sup>介绍了使用 PDFBox 包来提取 PDF 文件的文本,但是 PDFBox 对中文内容支持不佳,只能用来处理原义字符的文本流;文献<sup>[12]</sup>使用 iText 包中的 PdfReader 研究

如何将 MSOffice 格式的文件转换为 PDF 格式文件；文献<sup>[13]</sup>探讨了使用 Adobe 阅读器提供的 plug-in 插件进行 PDF 文件的操作；文献<sup>[14]</sup>探讨了如何使用 PDF2TXT 包和 PDF2HTML 包将 PDF 格式的文件转换为 txt 和 html 格式，然后再提取文本内容的思路。

而一些专门处理 PDF 文件的类库，如基于 C++ 的 XPDF、基于 Java 应用的 PDFBox 和 iText、基于 .Net 的 iTextSharp 和 PDFlib 等，仅仅用来生成 PDF 文件<sup>[15, 16]</sup>。专门用来解析 PDF 文件的类库比较少，或者存在一些问题，如对中文支持不友好，只用于 PDF 文件文本的提取等。

本文分析了 PDF 文件的格式：PDF 对象、PDF 文件物理结构、PDF 文件逻辑结构和 PDF 的内容流。提出了对 PDF 文件按章节进行切分的方法，当用户预览文件时，只需传输需求的章节，不用传输整个 PDF 文件，减少了文件传输量，降低了文件在线预览服务的响应时间。

### 1.2.3 负载均衡研究现状

服务器集群的概念起源于 20 世纪 90 年代，由美国的 UIUC 大学提出的“NVSA Scalable Web Server Cluster”为原型<sup>[17]</sup>，随后 Rice 大学和 IBM 公司等机构也开始研究服务器集群，推动了服务器集群的发展。

集群的核心技术在于负载均衡技术，国内外很多厂商在基于软件和基于硬件上推出了自己的负载均衡产品。如清华大学的 TH-Web Cluster，实现了可扩展的 Web 服务器集群；Cisco 的 Local Director 基于最小连接和快速响应算法，将用户的请求分配到响应速度最快的服务器；IBM 的 Network Dispatcher 通过选择当前连接最少量的服务器为请求的目标。

负载均衡技术主要分为静态负载均衡和动态负载均衡。静态负载均衡策略需要提前知道服务器的相关信息，从而提前定好参数，确定请求的分发策略；动态负载均衡策略能够考虑节点服务器的实时负载来分发请求，自动适应节点服务器的负载变化，改进了由于集群在运行时任务处理的不可预知，使得节点服务器的负载发生波动，在时间的推移下各个节点服务器的负载差别会变得很大，从而使得集群性能下降的问题。<sup>[18-21]</sup>

Nginx 由于高并发、内存占用率低和可扩展等优点，得到了国内外越来越多研究者的喜爱。如 Google 基于 Nginx 通过优化带宽传输和域名映射降开发了 ngx\_pagespeed 模块，提高了网站的加载速度；Nginx 官方开发的 Nginx Plus 商业版本对大规模集群带来了更好的解决办法；淘宝在 Nginx 的基础之上增加了动态编译的功能、改进 Nginx 的负载策略，其稳定性和高性能得到了验证；新浪在 Nginx 原有的缓存基础上，开发了



NCACHE 缓存系统,可以满足大并发、大存储的需求。[4, 22]

本文对 Nginx 默认的静态轮询算法做了改进,通过实时监测节点服务器的负载和修改 Nginx 的源代码来实时改变节点服务器的权重,更好的分发用户的请求。

综上所述,本文的主要研究方向为:对 MongoDB 的数据均衡策略加以改进,使得冷热数据可以均衡存储;分析 PDF 文件的格式,将 PDF 文件按章节切分和存储;研究 Nginx 的静态轮询算法,将其改进为根据节点服务器的负载动态更新权重。

### 1.3 本文的主要工作和创新点

通过深入的调查分析和查阅国内外相关学者在本研究领域的资料后,本文说明了课题的研究背景和意义,以及研究现状。在前人研究的基础之上本文将继续深入研究,主要完成的工作有:

1) 对现有的文件在线预览服务做了调查,由于 Flash 存在占用资源过高、安全性和可靠性低的问题,本文改变了以往依赖 Flash 进行文件预览的方式,而是将多种格式的源文件转换为 PDF 文件格式和 HTML 文件格式在线预览。[15]

2) 由于一些 PDF 文件过大,加上网络带宽的限制,用户对文件某一章节感兴趣时需要预览整个文件,造成用户等待时间过长,影响用户体验。所以本文研究了 PDF 文件解析,将 PDF 文件按章节切分、存储和传输,减少用户等待的时间。

3) 面对高并发的文件转换和切分请求,单个服务器处理受限。本文采用集群的方式来处理请求,通过修改 Nginx 的源代码,将 Nginx 默认的加权轮询算法(weighted round robin)改为节点服务器权重实时变化的动态加权轮询算法(dynamic weighted round robin)。

4) 改进了 MongoDB 的数据均衡策略,对 MongoDB 集群中的冷热数据均衡问题做了深入研究,提出了基于队列的“热数据”识别算法、数据块(Chunk)热权重计算和迁移策略,实现了冷热数据在节点服务器的均衡分布,提高了 MongoDB 集群的性能。

5) 分模块开发了文件在线预览服务的相关工具,降低了系统的耦合性和提高了系统的扩展性,并在高负载网络请求下对文件在线预览服务做了性能测试。

本文的创新点有:

1) 用户预览文件时,不用加载整个文件,而是根据服务器返回的文件目录,选择感兴趣的章节,定向预览,从而减少数据传输量,快速响应用户的预览请求。

2) 改进了 MongoDB 的数据均衡策略,在基于队列的“热数据”识别算法基础上,提出了数据块(Chunk)热权重计算方法和数据块迁移策略,提升了集群的整体性能,减少了文件读写所用时间。

## 1.4 论文组织结构

第一章是“绪论”，主要介绍了论文课题的研究背景和意义，总结了与本课题相关的国内外的研究现状，最后介绍了论文的主要工作和组织结构。

第二章是“相关基础知识介绍”，介绍了与论文课题相关的理论知识，包括了 PDF 文件的逻辑结构和内容解析、MongoDB 数据库的特点和自动分片机制、缓存技术和反向代理技术。

第三章是“文件在线预览服务需求分析和设计”，介绍了文件在线预览服务的需求分析，包括功能需求和性能需求、关键设计，包括文件转换模块、文件切分模块和文件存储模块、以及高负载下的优化设计，包括缓存和基于 Nginx 的动态反馈负载均衡服务器的设计。

第四章是“MongoDB 数据均衡策略研究”，对 MongoDB 原有的数据均衡策略进行了研究，指出其不足之处，然后提出了基于冷热数据均衡的改进策略。

第五章是“文件在线预览服务的实现”，主要实现了文件在线预览服务相关的文件上传、文件下载、文件预览、文件转换和切分工具，以及工具的展示。

第六章是“文件在线预览服务实验分析”，主要用来检验服务能否对负载进行合理调度，使得在高负载的网络请求下文件转换和切分工具的负载相对均衡，MongoDB 中热点数据的存储也能相对均衡。

论文的最后是总结和展望，主要总结了本文实现的文件在线预览服务的优势和不足，并对将来需要研究的工作做了阐述。

## 1.5 本章小结

本章介绍了论文课题的研究背景和意义，总结了与本课题相关的国内外的研究现状，最后介绍了论文的主要工作和组织结构。



## 第二章 相关基础知识介绍

### 2.1 PDF 文件

原有的文件在线预览技术，是将各种格式的文件转换为 SWF、图片的格式，在客户的浏览器端使用 Flash 进行预览，但是 Flash 存在占用资源过高、安全性和可靠性低的问题。从 MacOS 10 和 Safari 10 开始，苹果已经默认禁用 Flash 插件，并且 Adobe 即将在 2020 年停止和分发 Flash。<sup>[2]</sup>这使得一些用户将无法进行文件预览，故本文将不再使用 Flash 进行文件预览。

PDF 文件和其名字 Portable 一样具有跨平台性，能够在 Windows 系统、Linux 系统和 Mac 系统上具有统一的阅读体验<sup>[15]</sup>，而且 PDF 文件一般体积小，适合网络传输，所以本文将文件的格式转换为 PDF 格式进行预览，故此在这里简要介绍 PDF 的相关知识。

#### 2.1.1 PDF 文件逻辑结构

PDF 文件和 HTML、XML 等文件一样，属于结构化的文件，在逻辑上 PDF 文件的层次结构为树结构，每个父节点都有其对应孩子节点的引用，如图 2-1 所示为简单的 PDF 结构，树的根节点为 Trailer 对象，是 PDF 文件的入口，其子节点为目录节点(catalog)和其它信息节点(Other Info)，目录节点包含页面组对象(Pages)和大纲节点(Outlines)，页面组节点包含页面对象(Page)，页面对象中封装了各个页面显示的内容，如字体、图片、视频等，还有页面的布局信息，如页面的显示大小，文字位置等，以上的所有信息共同完成了对 PDF 页面的描述。

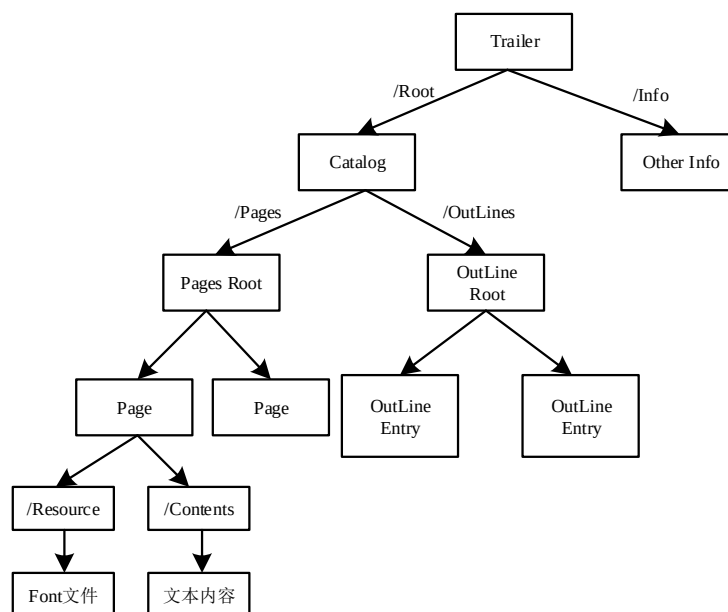


图 2-1 PDF 文件的逻辑结构

## 2.1.2 PDF 文件内容解析

PDF 文件的逻辑解析过程为：首先读取 PDF 文件的第一行来判断文件是否为 PDF 文件和相应的版本号，如果是 PDF 文件然后解析文件尾，获得根目录对象，同时完成交叉引用表的解析，获得 PDF 文件中各个对象的实际物理地址，然后对根目录进行深度优先遍历，获取页面组对象、页面对象，页面对象包含了页面布局和页面内容对象的引用。通过上面的解析过程从而获得 PDF 文件的全部信息。解析流程如图 2-2 所示。

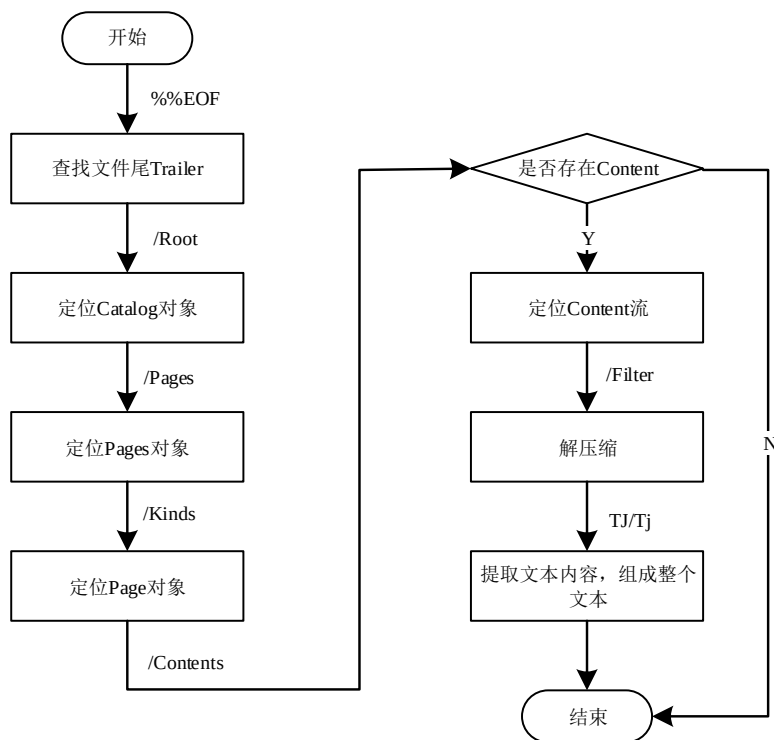


图 2-2 PDF 解析流程

## 2.2 MongoDB 数据库

MongoDB 是面向文档的、分布式的非关系型数据库(NoSQL)，由 C++编写，目的在于为 web 应用提供可扩展、高性能的数据存储方案<sup>[5, 6, 23]</sup>。MongoDB 是最接近关系型数据库的 NoSQL 数据库。在 JSON 数据格式上增加了更多的数据类型，使用数据类型更加丰富的 BSON 格式，用键值对(key => value)的方式将数据存储为文档<sup>[6]</sup>。MongoDB 中的基本概念有数据库(Database)、集合(Collection)和文档(Document)，与关系型数据库相比较，其中“文档”相对应于关系型数据库中的“行”，“集合”相对应于关系型数据库中的“表”。“文档”是 MongoDB 的基本数据单元，“集合”是 MongoDB 的文档组。

### 2.2.1 MongoDB 数据库特点

MongoDB 的特点是高性能、易扩展、易使用、接口丰富和存储数据方便高效，其

主要功能特性有<sup>[5, 8-10, 23-25]</sup>:

- 1) 使用集合存储数据。MongoDB 的数据先会分组, 然后存储在数据集中, 这种数据集称为集合。集合有唯一的标识名, 理论上可以包含无限数目的文档。
- 2) 模式自由。在集合中没有关系型数据库行和列的概念, 每个文档可以有属于自己的键, 因此可以根据实际情况方便的修改原有的存储结构。
- 3) 支持动态查询。SQL 数据库的查询方式基本支持, 有着丰富的查询接口。
- 4) 支持完全索引。MongoDB 的查询优化器通过分析查询的语句, 完善查询方式, 能够对文档的内嵌对象和数组进行索引。
- 5) 高效数据存储。使用 GridFS 来存储大二进制文件(图片、视频等), 在存储的时候不产生磁盘碎片。
- 6) 支持复制和故障恢复。MongoDB 的两种数据复制机制: 主从复制和副本集。其中副本集具有故障自动恢复的功能, 从而提高了数据的可用性和安全性, 降低了数据的维护成本。
- 7) 自动分片机制。MongoDB 支持自动分片, 实现了数据库的水平扩展, 可以动态添加额外的服务器。

### 2.2.2 MongoDB 自动分片机制

自动分片(Auto-Sharding)是 MongoDB 将数据拆分, 分散存储到不同服务器中的方法, 它是 MongoDB 水平无限扩容的方式<sup>[5, 9]</sup>。通过横向切分数据, 每台服务器只需负责一部分的数据, 从而降低了单台服务器的 CPU、内存和 I/O 的压力。这样就不需要性能很高的服务器就能支持大数据的存储和高负载请求的处理。

在自动分片方案中, 为了让每个分片中的数据均衡且高效迁移, MongoDB 根据片键(shard key)将分片中的数据拆分为更小的数据块(chunk)。同一个 collection 的数据不一定存储在同一个分片中, 但是一个数据块只能存储在一个分片中。图 2-3 为分片集群的结构。

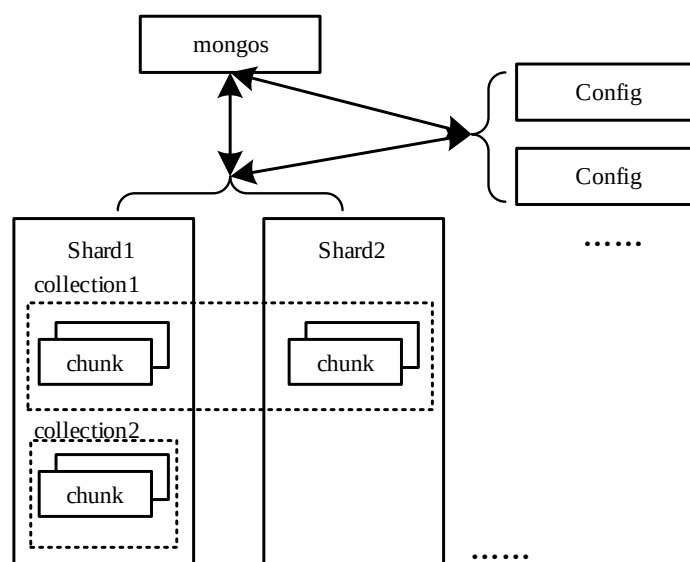


图 2-3 分片集群结构

由图 2-3 可知分片集群由 shard、Config 和 mongos 三个组件组成，各个组件的介绍如下：

### 1) mongos

mongos 组件相当于集群的路由器，是集群和客户端交互的窗口，它使集群中的多个组件看起来像单一的系统，客户端不需要关心数据在分片中是怎样存储。其功能是将读取和写入的请求从应用程序路由到具体的分片。在一个分片集群中可以存在一个或者多个 mongos，在一般的情况下多个的 mongos 能够缓解请求的压力。

### 2) Config

Config 为分片集群中的配置服务器，它存储了集群中的元数据(MetaData)，包含路由、分片和数据块的映射关系等，是集群的枢纽。因为 mongos 中不储存路由分片的信息，所以当 mongos 首次启动或者重启的时候都会向 Config 中加载集群的配置信息，而一旦配置服务器信息发生了变化，就会通知所有的 mongos 进行数据更新。为了保证集群中的元数据不丢失，一般都会配置多个 Config 服务器，每个 Config 服务器都含有一份现有集群的信息。

### 3) Shard

Shard 相当于 Hadoop HDFS 中的 dataNode，用来存储数据，是数据存储组件。集群中的每个 Shard 存储了整体数据中的部分数据，所以如果分片丢失就会造成整体数据的不完整，因此在实际的使用中，分片常常和副本同时使用，用来防止数据片的单点故障。在分片中最小的存储单位是数据块(chunk)，在数据均衡策略中就是通过迁移数据块，来使得系统的数据均衡分布在各个片中。

## 2.3 缓存技术

缓存技术是指将已经访问过的资源通过一定的方式保存在内存中,当资源再次被请求的时候,可以直接在内存中获取避免再次向服务器或者数据库发送 HTTP 请求,降低系统的响应时间,快速响应用户的请求。<sup>[26, 27]</sup>在系统设计中,可以将数据采用键值对的形式(key-value)将需要缓存的内容保存在内存中。

## 2.4 反向代理技术

反向代理(Reverse Proxy)是指用户的请求发送到代理服务器,然后代理服务器将请求转发到内部网络的服务器,并将服务器上得到的结果返回给用户。<sup>[4, 20, 22, 28, 29]</sup>

对于服务器集群,当面临高负载网络请求的时候,必须保证请求合理的分配到各个节点服务器上,这样才能高效处理用户的请求,提高系统的整体性能。否则,一些服务器分配的负载过大,使得这些节点的响应时间增加;而另一些服务器节点的负载很低,没有充分使用到节点服务器的计算资源。因此在高负载网络请求中,反向代理技术在系统中显得尤为重要。

## 2.5 本章小结

本章主要介绍了 PDF 文件的逻辑结构和 PDF 文件解析的流程,其次介绍了 MongoDB 数据库特点以及 MongoDB 能够实现水平扩展的自动分片机制,最后简略介绍了缓存技术和反向代理技术。这些相关知识在高负载网络请求的时候,能够减少系统的响应时间,提高系统的整体性能。

## 第三章 文件在线预览服务需求分析与设计

本章介绍了文件在线预览服务的需求分析，同时对系统关键部分的设计做了介绍，最后为了满足快速响应的需求，对系统的优化设计做了说明。

### 3.1 文件在线预览服务需求分析

#### 3.1.1 功能需求

文件在线预览服务主要实现在客户的浏览器端对文件进行在线预览，同时也支持用户进行单个文件或多文件上传的功能。

原有的文件在线预览技术，是将各种格式的文件转换为 SWF、图片的格式，在客户的浏览器端使用 Flash 进行预览，但是 Flash 存在占用资源过高、安全性和可靠性低的问题<sup>[2]</sup>，所以文件预览工具提供 PDF 的格式进行预览。为满足用户的基本需求，在预览的时候也提供打印、放大缩小和幻灯片放映的功能，文件预览工具基本的功能需求如下：

- 1) 文件上传功能：支持单个文件、多个文件和更新文件版本。
- 2) 文件下载功能：支持文件的下载。
- 3) 文件转换功能：支持 Microsoft Office 文件格式、图片、纯文本、PDF 文件格式和压缩文件的转换。
- 4) 文件预览功能

文件显示功能：在预览的时候支持缩略图和大图，在拖动滚动条和点击缩略图的时候能够跳转到对应的大图显示。

文件翻页功能：在预览的时候支持支持上一页、下一页和页面跳转的功能。

文件缩放功能：在预览的时候文件可以按比例缩放和还原到默认的比例。

文件打印功能：在预览的时候用户可以直接打印文件。

文件加载功能：在预览出现异常的时候，可以重新加载文件。

文件提示功能：在预览文件转换错误的时候，提示用户文件转换错误；大文件出现加载时间长的时候，提示用户文件正在加载等。

- 5) 日志服务功能：文件预览工具生成文件预览的日志，包含用户信息、文件信息和文件预览时间等。

- 6) 文件报错功能：文件由于未知的原因出现用户无法预览的情况，可以直接将错误信息发送给管理员。

由基本功能需求可知，在文件在线预览服务的过程中，文件预览工具的主要参与者是用户和系统的管理员。用户进行预览，在此期间可以打印文件、对文件进行页面的缩放和页面跳转，满足了用户预览时候的基本需求；系统管理员可以查看用户预览生成的日志文件和报错信息，并且处理报错信息。

系统的用例如图 3-1 所示。

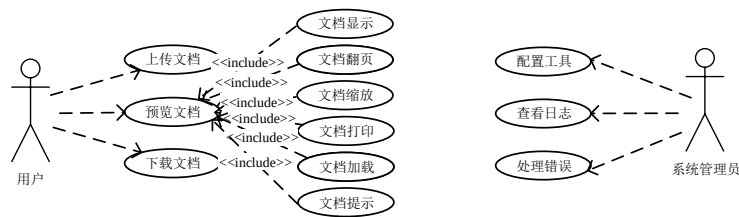


图 3-1 文件预览工具用例图

其中预览文件用例如表 3-1 所示。

表 3-1 预览文件用例描述

| 预览文件用例描述 |                                                                                            |
|----------|--------------------------------------------------------------------------------------------|
| 用例名称     | 预览文件                                                                                       |
| 简要说明     | 用户可以对文件进行预览，当预览成功时可以对文件进行缩放、打印等操作；预览失败时会提醒用户预览失败，并且发送失败信息给系统管理员。                           |
| 前置条件     | 用户进入自己的帮区资料柜，点击需要浏览的文件。                                                                    |
| 后置条件     | 无                                                                                          |
| 基本流程     | 1) 用户进入自己帮区资料柜，点击需要浏览的文件。<br>2) 如果用户对文件没有浏览的权限，则告诉用户“没有浏览文件的权限”。<br>3) 如果用户有权限，则显示文件的预览界面。 |
| 成功场景     | 用户能够成功预览文件。                                                                                |
| 失败场景     | 1) 用户没有权限预览文件。<br>2) 文件转换失败。<br>3) 需要转换的文件格式不持支。<br>4) 用户的网络不佳。<br>5) 其它情况。                |

### 3.1.2 性能需求

随着文件在线预览服务的发展，系统中的文件会越来越多，必须考虑好潜在的海量数据存储业务的需求和业务存储架构的扩展。本文的业务对象是科技工作者，据 2014 统计截止，全国科技工作者数量是 8100 万，预计到 2016 年底的数量可能要超过 1 个亿。<sup>[30]</sup>面对如此之多的用户所带来的高并发请求和海量文件的存储，如何快速响应是本文研究的重点。文件在线预览服务的性能需求为以下几个方面：

1) 准确性，在资料柜中由于存在多个用户上传和浏览文件，所以同一个文件可能存在多个不同的版本，当用户预览文件的时候，默认是预览最新版本的文件，保证用户预览的是最新的文件。同时，转换格式后的文件排版格式应该和原文件没有较大的差异，不能让用户对文件内容产生歧义。

2) 实时性，用户预览文件的时候应该可以实时加载文件，不用将要预览的文件全部传输到用户的浏览器，而是在用户预览的时候实时加载，快速响应用户的请求。

3) 安全性，用户只有具备上传、预览和下载的权限才能对文件进行操作，保证文件的安全性，在用户操作文件之前，系统会对用户的权限进行验证，判断用户是否有相应的权限。

4) 高性能，当文件在线预览服务面临高负载的网络请求的时候，系统能够高效处理请求，满足较高的并发用户数，减少响应时间，使用户有良好的体验。

5) 扩展性，文件在线预览服务应具有好的扩展性，能够作为一个独立的模块加入到其它需要预览功能的系统中。

响应时间是从发出 HTTP 请求，到收到 HTTP 响应，这个过程所经历的时间。响应时间在低负载网络请求和高负载网络请求下计算不同，分别为：

$$\begin{aligned}
 \text{低负载的响应时间} = & \text{客户端到 Web 服务器的网络延迟} * 2 & + \\
 & \text{请求数据量} / \text{数据发送速度} & + \\
 & \text{Tomcat 业务代码执行时间} & + \\
 & \text{Tomcat 到数据库的网络延迟} * 2 & + \\
 & \text{数据库执行查询所需时间} & + \\
 & \text{查询结果数据量} / \text{数据发送速度} & + \\
 & \text{响应数据量} / \text{数据发送速度} \\
 \text{高负载的响应时间} = & \text{客户机到 Web 服务器的网络延迟} * 2 & + \\
 & \text{请求数据量} / \text{数据发送速度} & +
 \end{aligned}$$



|                    |   |
|--------------------|---|
| 请求在 Tomcat 中排队的时间  | + |
| Tomcat 业务代码执行时间    | + |
| Tomcat 到数据库的网络延迟*2 | + |
| 查询请求在数据库中排队的时间     | + |
| 数据库执行查询所需时间        | + |
| 查询结果数据量 / 数据发送速度   | + |
| 响应数据量 / 数据发送速度     |   |

显然,在低负载的情况下,响应时间的可变部分与服务器的处理能力、请求数据量、响应数据量和业务代码、SQL 语句的复杂度有关,其它部分基本为常量;在高负载的情况下,响应时间在低负载的基础上增加了 HTTP 请求和 SQL 请求的排队时间。

文件在线预览服务中,通过减少响应数据量(不用加载整个文件,而是根据服务器返回的文件目录,选择用户感兴趣的特定章节)和 HTTP 请求、SQL 请求的排队时间(服务器集群)来减少响应时间,做到快速响应,具体方法将在下文介绍。

## 3.2 文件在线预览服务设计

文件在线预览服务的设计分为功能结构设计和性能优化设计两个部分。功能结构设计介绍了文件在线预览服务的整体架构和对应模块;性能优化设计介绍了文件在线预览服务如何高性能处理高负载网络请求,减少系统的响应时间。

### 3.2.1 功能结构设计

文件在线预览服务的整体架构由以下几个部分组成:文件上传工具、文件预览工具、文件下载工具、文件服务器、文件转换和切分工具和文件存储模块构成。各个部分的介绍如下:

- 1) 文件上传工具:负责将用户的文件上传到 Web Server,包含单个文件和多个文件上传。
- 2) 文件预览工具:用户预览文件。
- 3) 文件下载工具:用户用来下载源文件。
- 4) 文件服务器:负责调度文件转换请求,当用户上传文件的时候,文件服务器选择合理的节点服务器处理文件转换的业务;同时提供公共缓存空间,缓存预览过的文件。
- 5) 文件转换和切分工具:由文件转换模块和文件切分模块组成。文件转换模块负责将用户上传的其它格式的文件转换为 PDF 格式和 HTML 格式;文件切分模块解析 PDF

文件内容，将 PDF 文件按章节进行切分。

6) 文件存储模块：用来存储源文件和转换后的文件。

文件在线预览服务的总体架构如图 3-2 所示。

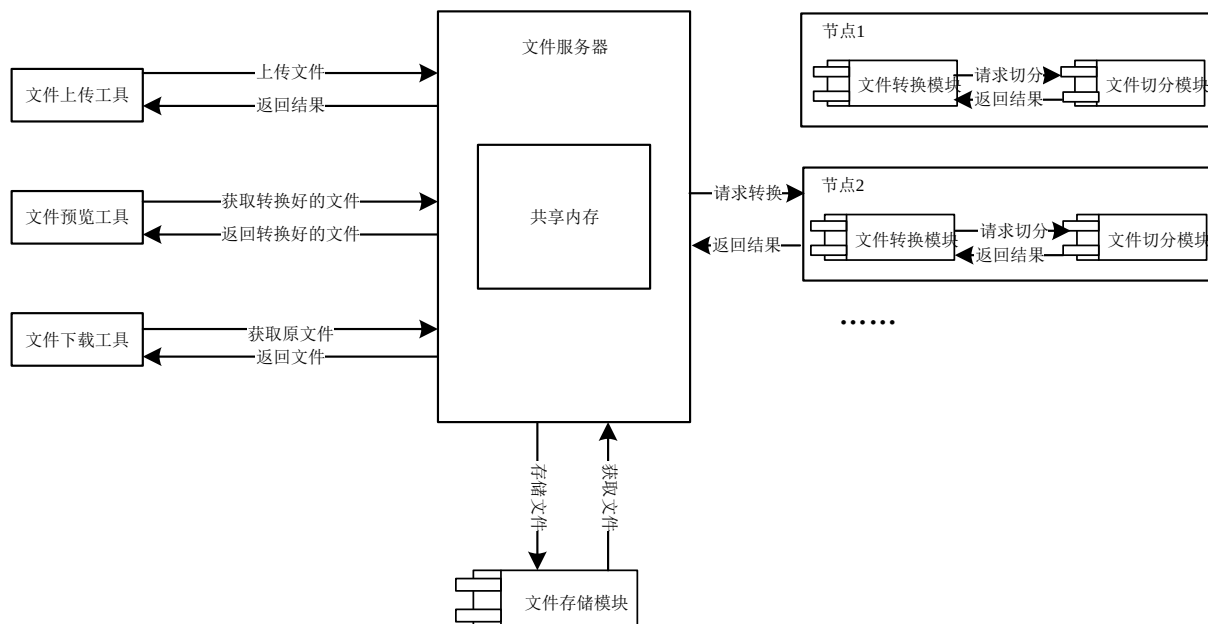


图 3-2 文件在线预览服务的总体架构

用户上传源文件之后，需要将各种格式的文件转换为 PDF 格式和 HTML 格式的文件，如果文件存在目录，则需要将 PDF 文件按目录进行切分，最后需要将源文件和转换后的文件存储。因此，本节着重介绍文件在线预览服务的功能结构设计：文件转换模块、文件切分模块和文件存储模块。

### 3.2.1.1 文件转换模块

文件转换模块负责将用户上传的文件进行转换，转换为适合用户在客户端进行预览的文件格式。

由第二章的相关介绍可知，因为 Flash 存在占用资源过高、安全性和可靠性低的问题<sup>[2]</sup>，而且 Adobe 即将在 2020 年停止和分发 Flash。所以本文舍弃以往文件预览系统的做法，不再使用 Flash 进行文件预览。而由于现在 PDF 文件的广泛使用和各浏览器都支持 PDF 格式的浏览，故本文将用户上传的文件转换为 PDF 格式的文件进行预览。因为移动端的有些浏览器不支持 PDF 文件的预览，为了提高用户体验，在此也将 PDF 格式转换为 HTML 格式在移动端进行预览。同时本文也支持压缩文件的在线预览。

单个文件的转换流程为：

1) 用户上传文件。

2) 根据上传文件的后缀,判断文件是否可以转换。如果是不可转换的类型,则直接结束,否则进入下一步。

3) 文件转换服务器在收到负载均衡服务器的转换请求后,将待转换的文件放到 uploadFile 文件夹,转换文件,并将转换为 PDF 格式的文件存放到 convertedFile 文件夹中。

4) 调用文件切分模块,将转换好的 PDF 文件按目录切分。

5) 将切分后的 PDF 文件转换为 HTML 格式。

文件转换的流程如图 3-3 所示。

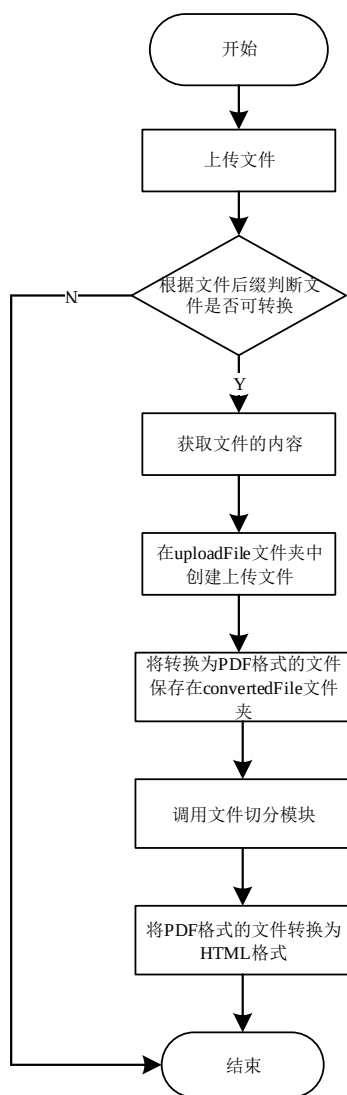


图 3-3 文件转换流程

随着用户需求的增加,用户不仅仅需要在线预览文件,而且需要在线预览压缩包(zip、rar、jar)文件,因此在线预览服务也支持压缩包的预览。

压缩包的转换流程为:

- 1) 用户上传压缩包。
  - 2) 根据上传压缩包文件的后缀, 判断压缩包是否可以转换。如果是不可转换的类型, 则直接结束, 否则进入下一步。
  - 3) 文件转换服务器在收到负载均衡服务器的转换请求后, 将待转换的压缩包放到uploadFile 文件夹, 然后将压缩包在文件夹中解压, 遍历解压后压缩包中的文件。
  - 4) 在遍历文件的过程中转换文件, 其过程和单个文件转换一样。
- 压缩包转换流程如图 3-4 所示。

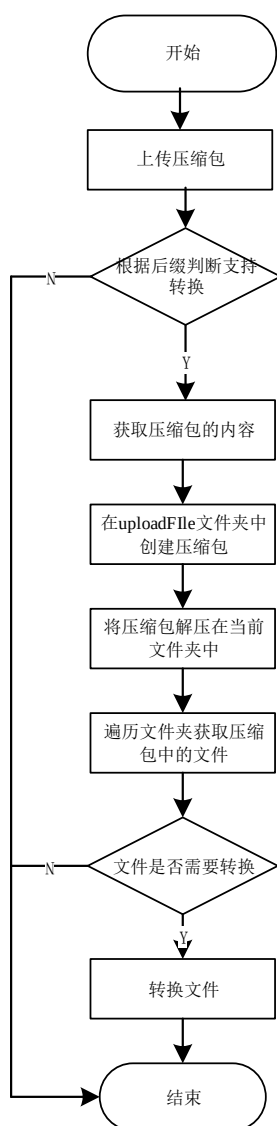


图 3-4 压缩包转换流程

### 3.2.1.2 文件切分模块

由于带宽的影响, 大文件在网络中传输耗时较长, 如果直接把整个文件传输给用户, 会使得用户的等待时间过长, 影响用户体验。而本文件在线预览服务主要针对科研工作

者，他们预览的文件主要是论文和书籍，可能只需要预览关键性的章节，对文件其它部分不太关心。如果将整个文件发给用户，用户需要等待很长时间。所以将文件按章节进行切分，减小单个文件的体积，同时只传输用户想要浏览的部分，能够有效提高用户体验。

文件切分的流程为：

- 1) 文件切分模块由文件转换模块调用，用来按章节切分 PDF 文件。
- 2) 文件切分模块首先判断 PDF 文件是否有大纲，如果有，则切分文件；如果没有，则不切分文件。
- 3) 文件转换模块将原 PDF 文件和切分好的章节文件(如果 PDF 文件可切分)返回给文件服务器。

文件切分流程如图 3-5 所示。

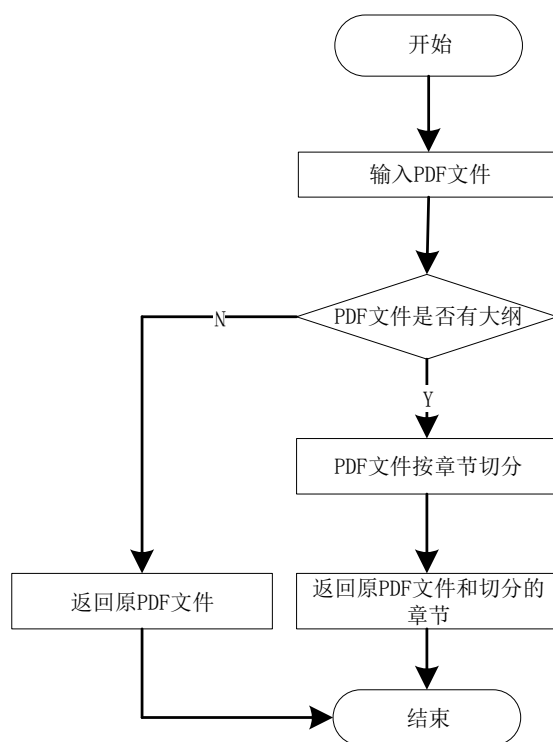


图 3-5 文件切分流程

### 3.2.1.3 文件存储模块

由于 MongoDB 与 SQL 数据库相比较有更加丰富的增、删、改、查接口，并且支持自动分片(auto-sharding)和副本集，在数据备份和恢复上更加简单便捷。MongoDB 的 BSON 文档格式和 Grid FS 对大文件的存储，满足文件在线预览服务对文件存储的需求。所以结合实际的应用场景，本文将使用 MongoDB 数据库存储文件。

文件存储在 files 集合中，其数据结构如表 3-2 所示。

表 3-2 文件集合 files

| 字段名称           | 字段类型           | 用途说明     |
|----------------|----------------|----------|
| storageID      | String         | 文件的唯一 ID |
| fileName       | String         | 文件名称     |
| fileLength     | 64-bit Integer | 文件长度     |
| uploadTime     | Timestamp      | 文件上传时间   |
| fileContent    | 内嵌文档           | 文件内容     |
| userID         | String         | 用户 ID    |
| belongedBandID | String         | 所属帮区     |
| belongedOrgID  | String         | 所属机构     |

### 3.2.2 性能优化设计

由于文件在线预览服务使用的场景广泛，往往需要处理大量的文件预览请求，如何高效的对并发请求进行处理，降低系统的响应时间是设计的关键。本文主要使用了缓存、基于 Nginx 的动态加权轮询算法和基于 MongoDB 数据均衡来提高系统的整体性能。前两种方法在本节详细介绍，后一种方法将在第四章介绍。

#### 3.2.2.1 缓存

文件在线预览服务应用了缓存技术。当用户发起一次预览请求的时候需要从数据库获取文件的相关信息，例如：文件内容、文件标题、文件类型、文件大小和是否加密等信息。如果再次请求同一份文件的时候，又需要和调用相关的接口，这会增加服务器的压力。为此，可以对文件及其相关信息进行缓存，即在文件服务器上建立一个公共的缓存区，通过文件的唯一标识 storageID 和文件的相关信息及内容流成对存入其中。当同一个文件请求时，可以避免调用相关的接口，直接返回预览文件，减少系统的响应时间，达到快速响应的目的。

但是文件服务器的内存空间有限，不能无限的将文件及其相关信息缓存在内存中，所以在有限的空间中如何更新缓存将直接影响文件查找的命中率。将不需要缓存的文件及时删除并释放相应的缓存空间，能够有效利用缓存区，提高系统的性能，目前常用的缓存管理算法为<sup>[33]</sup>：

- 1) 最近最不常用(Least Frequently Used, LFU)将直接删除缓存区中访问频率最低的文件，从而保留缓存区中访问频率较高的文件。
- 2) 最近最少使用(Least Recently Used, LRU) 根据数据的访问记录来删除数据，基

本思想是：最近访问过的数据将来也会访问，长时间未被访问过的数据将来也不会访问，直接从缓存中删除。

文件服务器对缓存的文件采用“最近最不常用”(Least Frequently Used, LFU)淘汰方法，避免缓存的内容太多，影响文件服务器的性能。虽然“最近最少使用”(Least Recently Used, LRU)也是一种常用的缓存替换算法，但是 LRU 效率没有 LFU 高，并且 LFU 能够避免周期性或者偶发性的操作导致缓存命中率下降的问题。如图 3-6 是缓存信息的结构，在文件缓存中增加文件访问频率的标识，文件每次被命中访问的时候，访问频率加 1，每次到更新的时间或者缓存空间不足时将访问频率最少的文件删除。

```
class CachedFile {
    String storageID;
    String fileName;
    File value;
    String belongedBandID;
    String belongedOrgID;
    int accessFrequency;
}
```

图 3-6 缓存信息结构

### 3.2.2.2 基于 Nginx 的动态加权轮询算法

当面临大量用户上传文件的时候，单个文件转换和切分服务器不能满足用户的请求，为此使用集群的方式来处理文件转换和切分。本节介绍基于 Nginx 的动态加权轮询算法，用来处理高负载的网络请求。

Nginx 默认的负载均衡算法是加权轮询算法(weighted round robin)<sup>[34]</sup>，通过预先设置各个服务器的权重来处理请求，这很难反应每台服务器的真实性能，对此，本文在原算法的基础之上，加以改进，实时更新各个服务器节点的权重。

#### 1) 服务器的默认权重

主要参考服务器节点以下方面信息：CPU 主频、CPU 核数、内存容量和网络带宽，对应单个服务器节点  $S_i (1 \leq i \leq n)$ ， $A_{CPU\_fre}(S_i)$  为 CPU 主频、 $A_{CPU\_cores}(S_i)$  为 CPU 核数、 $A_{memory}(S_i)$  为内存容量、 $A_{network}(S_i)$  为带宽定义。对于服务器集群的整个计算能力，其表达式分别为：

$$C = \sum_{i=1}^n (A_{CPU\_fre}(S_i) * A_{CPU\_cores}(S_i))$$

$$M = \sum_{i=1}^n A_{memory}(S_i)$$

$$N = \sum_{i=1}^n A_{network}(S_i)$$

在集群第一次启动的时候将所有节点的性能进行求和,然后计算各个节点性能占总性能的比重。性能越好的服务器权重越高,所以被分配的任务也越多。初始化的时候每台服务器权重的定义为 $I(S_i)$ ,利用公式(3.1)计算每个服务器节点的初始权重:

$$I(S_i) = \lceil K * (\frac{A_{CPU\_fre}(S_i) * A_{CPU\_cores}(S_i)}{C} + \frac{A_{memory}(S_i)}{M} + \frac{A_{network}(S_i)}{N}) \rceil \quad (3.1)$$

其中  $K$  是一个常数(本文  $K=10$ ),用来调整节点权重大小的倍数,通过将  $I(S_i)$ 调整为一个较大的整数,使得在舍弃小数部分的时候,误差更小。

## 2) 服务器当前负载值的计算

主要参考服务器节点以下方面信息:CPU 利用率、内存利用率、网络带宽占有率和磁盘 I/O 占用率。对应单个服务器节点  $S_i(1 \leq i \leq n)$ ,其资源利用率的定义为  $L(S_i)$ ,利用公式(3.2)计算单个服务器节点的资源利用率:

$$L(S_i) = k_1 * L_{CPU}(S_i) + k_2 * L_{memory}(S_i) + k_3 * L_{network}(S_i) + k_4 * L_{disk}(S_i) \quad (3.2)$$

其中  $L_{CPU}(S_i)$ 为CPU利用率、 $L_{memory}(S_i)$ 为内存利用率、 $L_{network}(S_i)$ 为网络带宽占有率、 $L_{disk}(S_i)$ 为磁盘 IO 占有率,各项指标的取值范围在 $[0, 1]$ ;  $k_i(1 \leq i \leq 4)$ 表示各项指标的权重系数,满足的条件为:  $k_i \geq 0 (1 \leq i \leq 4)$ 而且  $\sum_{i=1}^4 k_i = 1$ 。

根据实践的经验,在文件转换和切分的时候,节点服务器在满载时候性能瓶颈大多数的情况在 CPU,所以上述权重系数设置如下:

$$k_1 = 0.6$$

$$k_2 = 0.2$$

$$k_3 = 0.1$$

$$k_4 = 0.1$$

此时公式(3.2)为:

$$L(S_i) = 0.6 * L_{CPU}(S_i) + 0.2 * L_{memory}(S_i) + 0.1 * L_{network}(S_i) + 0.1 * L_{disk}(S_i) \quad (3.3)$$

只需要获取公式(3.3)涉及到的指标的具体值,就可以计算单个服务器节点的资源利用率  $L(S_i)$ 。服务器节点  $S_i$ 的实际负载越大时,其负载值  $L(S_i)$ 越接近 1;实际负载越小时,负载值  $L(S_i)$ 越接近 0。

## 3) 服务器动态权重计算

使用  $W(S_i)$ 表示服务器节点使用过程中的权重,对于集群中的任何一个服务器节点  $S_i$ ,其动态权重的公式为:

$$W(S_i) = [(1 - L(S_i)) * I(S_i)] \quad (3.4)$$

## 4) 更新服务器权重



在集群启动的时候, Nginx 收集节点的 CPU 主频、CPU 核数、内存容量和网络带宽, 然后根据公式(3.1)计算节点初始化权重 $I(S_i)$ ; 服务器节点收到来自 Nginx 的采集负载信息请求之后, 获取当前节点的负载信息, 包括: CPU 的利用率、内存利用率、网络带宽占用率和磁盘 IO 占用率, 然后通过公式(3.3)计算单个服务器节点的资源利用率 $L(S_i)$ , 最后根据公式(3.4)计算服务器节点新的权重 $W(S_i)$ 。

其中 CPU 的使用情况在“/proc/stat”文件中可以找到, 内存使用情况在“/proc/meminfo”文件中可以找到, 网络带宽在“/proc/net/dev”文件中可以找到, 磁盘 IO 通过命令: “iostat -d -x”得到。表 3-3 和表 3-4 分别列出了获取节点硬件配置和负载信息涉及到的文件位置、参考命令和相关接口。

表 3-3 节点硬件和负载信息获取方法

| 信息名称      | 涉及文件位置        | 参考指令                                       |
|-----------|---------------|--------------------------------------------|
| CPU 主频    | /proc/cpuinfo | cat /proc/cpuinfo                          |
| CPU 核数    | /proc/cpuinfo | cat /proc/cpuinfo  grep "processor"  wc -l |
| 内存总容量     | /proc/meminfo | cat /proc/meminfo  grep MemTotal uniq      |
| 网络带宽      | -             | ethtool 网卡名称                               |
| CPU 使用率   | /proc/stat    | cat /proc/stat                             |
| 内存使用率     | /proc/meminfo | cat /proc/meminfo                          |
| 磁盘 IO 使用率 | -             | iostat -d -x                               |
| 网络带宽使用率   | /proc/net/dev | cat /proc/net/dev                          |

表 3-4 获取硬件配置及负载信息的相关接口

| 方法名称                | 描述          |
|---------------------|-------------|
| getCpuFreq ()       | 获取 CPU 主频   |
| getCpuCoreNum ()    | 获取 CPU 核数   |
| getMemSize ()       | 获取内存总容量     |
| getBandwidth ()     | 获取网络带宽      |
| getCpuUsage ()      | 获取 CPU 使用率  |
| getMemUsage ()      | 获取内存使用率     |
| getDiskIoUsage ()   | 获取磁盘 IO 使用率 |
| getBandwidthUsage() | 获取网络带宽使用率   |

表 3-4 获取硬件配置及负载信息的相关接口（续）

|               |          |
|---------------|----------|
| getDevInfo()  | 获取硬件配置信息 |
| getLoadInfo() | 获取当前负载信息 |

### 3.3 本章小结

通过对文件在线预览服务的系统需求和性能需求进行分析，明确了文件在线预览服务的基本要求，然后对关键设计进行了介绍，并解决了在设计中遇到的问题，最后针对文件在线预览服务在面临高负载网络请求的场景时，通过缓存和增加基于 Nginx 的动态反馈负载均衡服务器来初步处理请求。

## 第四章 基于 MongoDB 数据均衡策略研究

通过第三章文件存储模块的介绍,本文采用 MongoDB 存储数据。如何高效、均衡的存储数据是本章节研究的重点。通过合理的数据均衡策略,充分利用服务器的资源,从而提升文件在线预览服务的整体性能。本章节对 MongoDB 的数据均衡策略进行了研究,指出其存在的问题,并提出了相应的优化方法。

### 4.1 MongoDB 数据均衡策略及问题分析

数据均衡策略是分布式系统实现的基础,如何在各个节点上使数据均衡分布,将直接影响系统的运行效率。在 MongoDB 中随着数据量的增加, MongoDB 会采用自动分片(Auto-Sharding)机制横向扩展数据<sup>[5]</sup>。MongoDB 的自动分片机制是将 collection 中的数据进行分割,然后将数据均衡存储在不同的分片服务器中。但是 MongoDB 仅仅是分片间数据量的均衡,不是数据访问热度的均衡,无法发挥集群真正的优势,由此本文对 MongoDB 原有的数据均衡策略进行研究和提出改进的办法。

#### 4.1.1 数据块(chunk)的拆分

在新分片的集合中刚开始的时候只有一个数据块,所有的文档都存在这个数据块上,因此数据块的文档范围是 $[-\infty, +\infty]$ 。随着数据块中数据的增加, MongoDB 将数据块自动分为两个块,采取的是“均分数据量”策略,这两个块的数据范围分别是 $[-\infty, n]$ 和 $[n, +\infty]$ ,其中  $n$  为某个数值<sup>[6]</sup>。

mongos 实时记录每个数据块的数据量大小,一旦数据量达到了某一个阈值,就会对数据块进行拆分,同时在 Config 服务器更新数据块的基本信息。在拆分的过程中遵循两个原则:首先是拆分点的选择要合适,不能将片键(shard key)相同的数据拆分到不同的 chunk 中;其次是拆分后的数据块所占的空间基本相等。当 mongos 发现数据块的大小(ChunkSize)超过了设置的阈值(MaxChunkSize)的时候,分片服务器就会拆分此数据块。拆分流程如图 4-2 所示。首先读取数据块包含的数据条数(ChunkCount),同时计算每条数据的平均(avgSize)大小:

$$\text{avgSize} = \text{ChunkSize} / \text{ChunkCount} \quad (4.1)$$

然后估计拆分之后每个数据块所包含的数据条数(subChunkCount):

$$\text{subChunkCount} = \text{MaxChunkSize} / (2 * \text{avgSize}) \quad (4.2)$$

通过索引找到数据块中 subChunkCount 的位置,从此位置开始将数据块拆分为两个大小相等的数据块。拆分流程如图 4-1 所示。

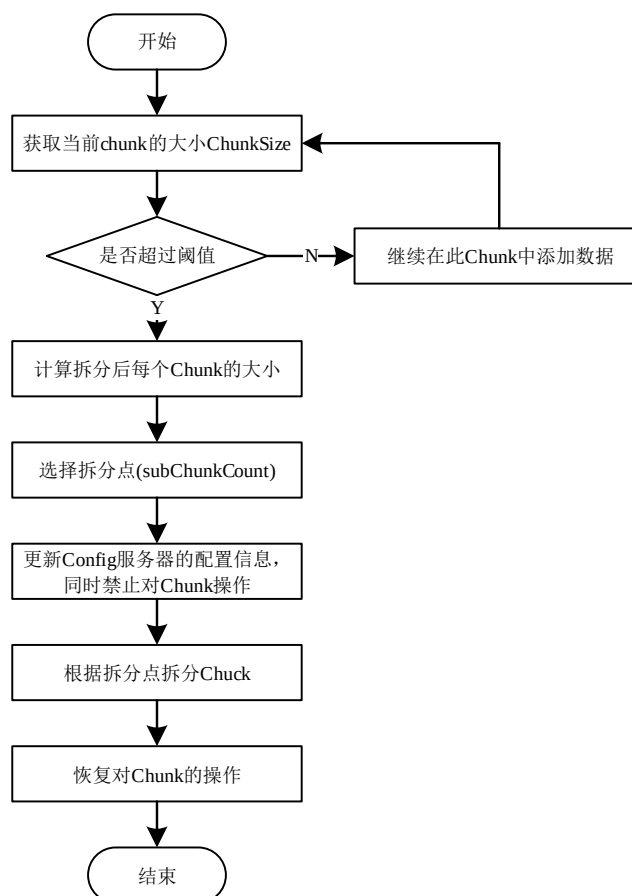


图 4-1 数据块拆分流程

### 4.1.2 数据均衡策略

随着 MongoDB 中数据量的不断增加, 每个分片中数据块的数量也会不断增加, 每个分片间数据块的数量可能存在差异。然而, 数据块多的服务器负载大, 数据块少的服务器负载小, 影响了系统的整体性能。为此需要将各个分片之中的数据块均衡, 把拥有较多数据块分片中的数据块迁移到拥有数量较少数据块的分片上。

MongoDB 支持自动的数据均衡策略, 将数据块均匀分散到各个分片上。其机制是假设数据块的数量和分片的负载成正比例, 集群负载失衡的表现就是一个分片比其它分片拥有更多数量的数据块。[5, 8-10, 23-25, 31]比如, 分片 A 拥有 100 个数据块, 分片 B 拥有 10 个数据块, 而 A 和 B 服务器的性能差不多, 因此集群的负载已经失衡, 那么 MongoDB 就会启动均衡器对数据进行迁移, 直到分片 A 和分片 B 上的数据块数量的差值小于数据迁移的条件, 即系统默认的迁移阈值。

MongoDB 的均衡器会周期性的检查各个分片间数据块数量的差值是否满足迁移阈值, 如果达到了, 均衡器就会开始对数据块迁移。均衡器会从数据块较多的分片上选择一个数据块, 在迁移之前会询问该片是否需要拆分, 如果需要拆分, 则拆分之后将数据

块迁移到块数量较少的分片上。其流程如图 4-2 所示。

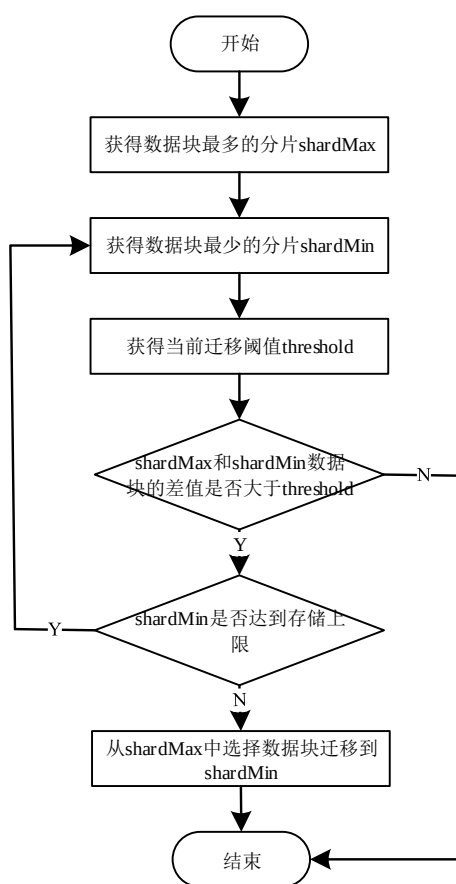


图 4-2 数据均衡算法流程

### 4.1.3 数据均衡策略不足

MongoDB 采用基于每个分片中数据块数量均衡的策略，通过数据块在分片间的迁移保证集群中各个分片服务器存储和维护的块数据量达到相对的平衡，来提升系统的整体性能。但是在现实的存储环境中数据的访问呈现相当高的局部性，根据访问频率的不同，频繁访问高的数据为“热数据”，很少或者从不访问的数据为“冷数据”。MongoDB 的数据均衡策略没有让冷热数据在分片中均匀分布，有可能让“热数据”存储在某一个分片中，但是各个分片的数据量均衡，使得此分片的负载高，而其它分片的负载低，发挥不了分布式存储的优势，而且由于“木桶效应”，如果一个服务器的负载急剧增加，尽管其它服务器正常运行，系统的整体性能也会急剧下降。为了解决冷热数据分布不均的问题，本文将在下一节提出基于队列的热数据识别算法。

## 4.2 改进的数据均衡策略

通过上一节对 MongoDB 数据均衡算法的研究，本节提出改进的算法，将集群中的“热数据”均衡存储，提高系统的整体性能。

### 4.2.1 基于队列的热数据识别算法

冷热数据识别在存储器有广泛的使用，因为存储器中存储颗粒的擦写次数有限，对数据进行冷热分类，从而将访问频率高的“热数据”存储在擦写次数较少的颗粒中；将访问频率低的“冷数据”存储在擦写次数较多的颗粒中，均衡存储器中存储颗粒的擦写次数，使得存储器的寿命得到延长。

存储器中早期的冷热数据分类算法是 two-level LRU 算法。该算法由台湾大学的张立平教授首次提出<sup>[35]</sup>，其基本思想是：两个固定长度的队列存储数据，其中一个队列 A 存放“热数据”，另一个队列 B 存放“候选热数据”，这两个队列都采用 LRU 算法进行数据更新，算法流程为：

- 1) 如果一个数据没有在队列 A 和队列 B 中出现过，则将数据放入候选队列 B。
- 2) 如果一个数据只在队列 B 中存在，则将数据存放在“热数据”队列 A 中。
- 3) 在“热数据”队列 A 中存在的标记为“热数据”，否则为“冷数据”。

two-level LRU 算法的结构如图 4-3 所示。

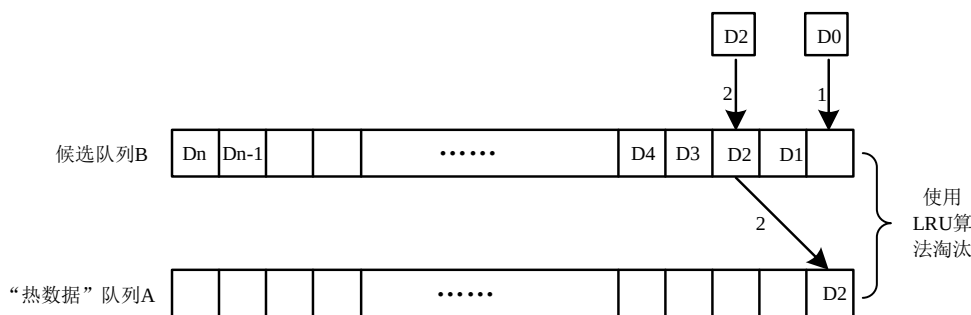


图 4-3 two-level LRU 算法

two-level LRU 算法的好坏取决于队列 A 和队列 B 的大小，一般情况下队列越长则算法性能最优。但是 two-level LRU 算法的核心是基于数据访问频率的冷热数据分类算法，无法根据数据访问时间区分冷热程度。文件在线预览服务中，刚刚上传的文件访问的频率越高；文件在系统中存在时间越久，被访问的频率越低，因此文件访问时间也是冷热数据分类的因素。比如，时间段 $[t_0, t_1]$ 中数据 D1 被访问了 8 次，数据 D2 被访问了 2 次；时间段 $[t_1, t_2]$ 中数据 D1 被访问了 2 次，数据 D2 被访问了 8 次，虽然在时间段 $[t_0, t_2]$ 内数据 D1 和 D2 都被访问了 10 次，根据 two-level LRU 算法，数据 D1 和数据 D2 都为热数据，但是在实际的文件预览过程中，对于  $t_2$  时刻，数据 D2 更应该作为“热数据”。因此在冷热数据分类算法中，应该能够展现数据的热度随时间的变化而变化。

为了有效反映数据访问次数随时间的变化关系，结合 two-level LRU 算法思想，本

文提出基于队列的热数据识别算法，其思想是使用一个队列来统计数据的权重，同时对队列中的元素采用先进先出(FIFO)的规则进行淘汰。算法结构如图 4-4 所示，实现方法如下：

- 1) 初始化一个长度为  $n$  的队列，为队列的每个位置分配一个权重(规则是队头权重为  $n$ ，队尾权重为 1，权重值依次递减)。
- 2) 当数据块被访问时，将其从队头插入队列中，其权重为  $n$ 。
- 3) 在判断某个数据块(Chunk)是否为“热数据”时，只需要扫描队列中的数据，将队列中相同数据块的权重相加，数值和越大，说明数据块在这段时间被访问的越频繁；数值和越小，说明数据块在这段时间被访问的越少。

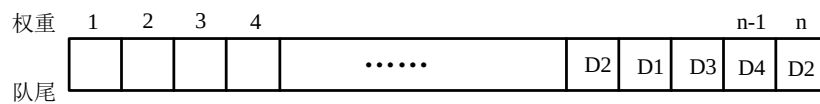


图 4-4 基于队列的热数据识别算法

该算法与上文介绍的 two-level LRU 算法相比较，综合考虑了数据访问时间和访问频率两种因素，同时也简化了算法的复杂度。算法中定长队列长度  $n$  的取值决定了冷热数据分类的准确性， $n$  的取值越大，算法的准确性越高，但是消耗的内存也越多。

#### 4.2.2 数据块热权重的计算

检查队列中是否有相同的数据块，然后将相同数据块的权重相加，即可得到数据块的热权重。假设队列的长度为 10，则队列如图 4-5 所示。

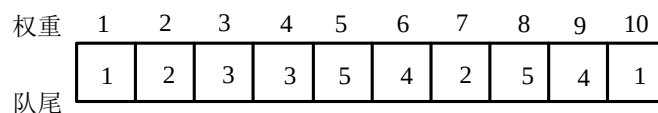


图 4-5 更新前队列状态

图中数据块 5 的热权重为  $5+8=13$ ；数据块 4 的热权重为  $6+9=15$ ；其它数据块的权重如表 4-1 所示。

表 4-1 更新前数据块权重

| 数据 ID | 1  | 2 | 3 | 4  | 5  |
|-------|----|---|---|----|----|
| 热权重   | 11 | 9 | 7 | 15 | 13 |

当数据块 5 再被访问时，队列更新如图 4-6 所示。

|    |   |   |   |   |   |   |   |   |   |    |
|----|---|---|---|---|---|---|---|---|---|----|
| 权重 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 队尾 | 2 | 3 | 3 | 5 | 4 | 2 | 5 | 4 | 1 | 5  |

图 4-6 更新后队列状态

此时数据块的权重如表 4-2 所示。

表 4-2 更新后数据块权重

|       |   |   |   |    |    |
|-------|---|---|---|----|----|
| 数据 ID | 1 | 2 | 3 | 4  | 5  |
| 热权重   | 9 | 8 | 5 | 13 | 21 |

### 4.2.3 数据块迁移策略

本文利用“基于队列的热数据识别算法”对 MongoDB 数据均衡算法进行改进，能够知道每个数据块的热权重，从而可以计算出每个分片的热权重。而 MongoDB 原有的数据迁移策略不能满足“均分热权重”的要求，因此需要重新研究新的数据块迁移策略。

本文提出的数据迁移策略伪代码为：

---

#### 数据块迁移策略

---

输入：分片的集合：shards，

节点服务器资源利用率阈值： $\alpha$ ，

输出：源分片 from

终分片 to

```

Function moveChunk(shards,  $\alpha$ ){
    minShards  $\leftarrow \emptyset$ ; // 初始化最小分片集合 minShards
    for shard in shards{
        // 获取单个分片的信息和所在节点服务器的资源利用率
        shardInfo = getShardInfo(shard);
        shedload = getLoad(shard);
        // 分片存储空间未耗尽且资源利用率小于  $\alpha$ ，则加入 minShards
        if(!isDraining(shardInfo) & shardLoad <  $\alpha$ )
            minShards.add(shard);
    }
    // 如果所有节点资源利用率小于  $\alpha$ 
    if(len(minShards)==len(shards)&maxChunks(shards)-minChunks(shards)>8)
    {
        shard = shards.maxChunkNums;
        chunk = shard.maxChunkWeight ();
        from  $\leftarrow$  chunk, to  $\leftarrow$  shards.minChunkNums;
        // 对 minShards 根据资源利用率从小到大排序
    }else{minShards = sorted(minShards);

```

---



```

if(shard.shardLoad  $\geq \alpha$ ){
    chunk = shard.maxChunkWeight();
    from  $\leftarrow$  chunk, to  $\leftarrow$  minShards.get();
}
}

```

根据实践经验,当节点服务器的资源利用率在 70%左右的时候<sup>[36]</sup>,节点服务器的资源既能得到充分利用,又有较短的响应时间。所以本文将数据块的迁移阈值设置为 70%,通过公式(3.2)可以得到节点服务器的当前资源利用率。数据块的迁移策略如图 4-7 所示,

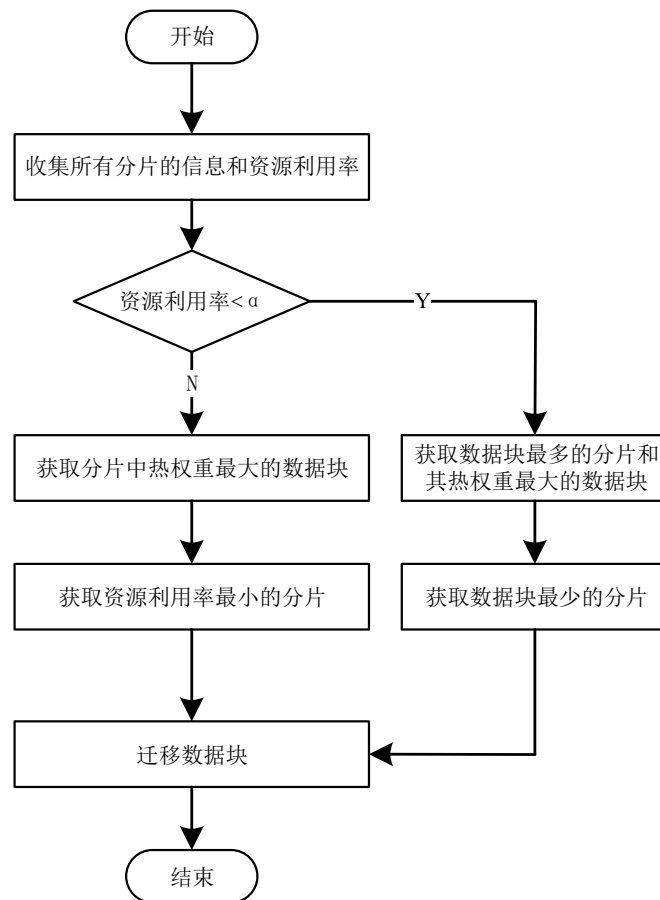


图 4-7 数据块迁移流程图

数据块迁移的具体步骤为:

- 1) 收集所有分片的信息和对于节点服务器的资源利用率。
- 2) 如果资源利用率小于 70%, 查找数据块最多和最少的分片  $\text{maxChunkNums}$  和  $\text{minChunkNums}$ ; 否则跳到第(5)步。
- 3) 在  $\text{maxChunkNums}$  中查找最大热权重的数据块  $\text{maxChunkWeight}$ 。
- 4) 将  $\text{maxChunkWeight}$  迁移到  $\text{minChunkNums}$  分片中。
- 5) 查找资源利用率大于 70%分片中的最大热权重数据块  $\text{maxChunkWeight}$ 。

将 maxChunkWeight 迁移到最小分片集合 minShards 中的分片中。

### 4.3 本章小结

本章首先介绍了 MongoDB 已有的数据均衡策略，指出了 MongoDB 采取“均分数据量”策略的不足，然后提出了改进的数据均衡策略，分别对集群中热数据块做了识别、数据块热权重的计算和数据块的迁移做了研究，从而提升了集群的整体性能。

## 第五章 文件在线预览服务的实现

本章通过开发文件在线预览服务的各个模块，实现文件在线预览服务的相关功能，最后通过各个工具的展示来验证文件在线预览服务的完成。

### 5.1 开发环境

本文在 CentOS 系统下完成开发的工作，考虑到程序的可移植性，本文主要使用 Java 编程语言。具体的开发环境如表 5-1 所示。

表 5-1 开发环境

| 名称      | 开发环境              |
|---------|-------------------|
| 操作系统    | CentOS 7          |
| 服务器     | Tomcat 7.0        |
| 数据库     | MySQL 5.7         |
| JDK     | JDK 1.7.0         |
| MongoDB | MongoDB 3.4.8     |
| 开发工具    | MyEclipse         |
| 编程语言    | Java/JavaScript 等 |

### 5.2 文件在线预览服务的开发和实现

文件在线预览服务的开发秉承“高内聚，低耦合”的原则，使得各个模块能够独立完成自己的功能，使其“重用性”和“移植性”大大增强。为此在开发的过程中让各个模块以工具的形式实现，其具体实现如下。

#### 5.2.1 文件上传工具

文件上传工具的前端采用 JavaScript 和 css3 来实现，后端采用 Java 实现。主要功能是完成单个文件上传、多个文件上传和新增文件版本，文件上传工具的主要流程为：

- 1) 用户点击“上传”按钮。
- 2) 判断用户有无上传权限：如果有，则继续；如果无，则结束。
- 3) 获取用户上传的文件，使用 Blowfish 加密用户文件，并保存到文件服务器。
- 4) 调用公共服务器的 3pa 接口，创建文件对象。

文件上传工具流程图如图 5-1 所示。

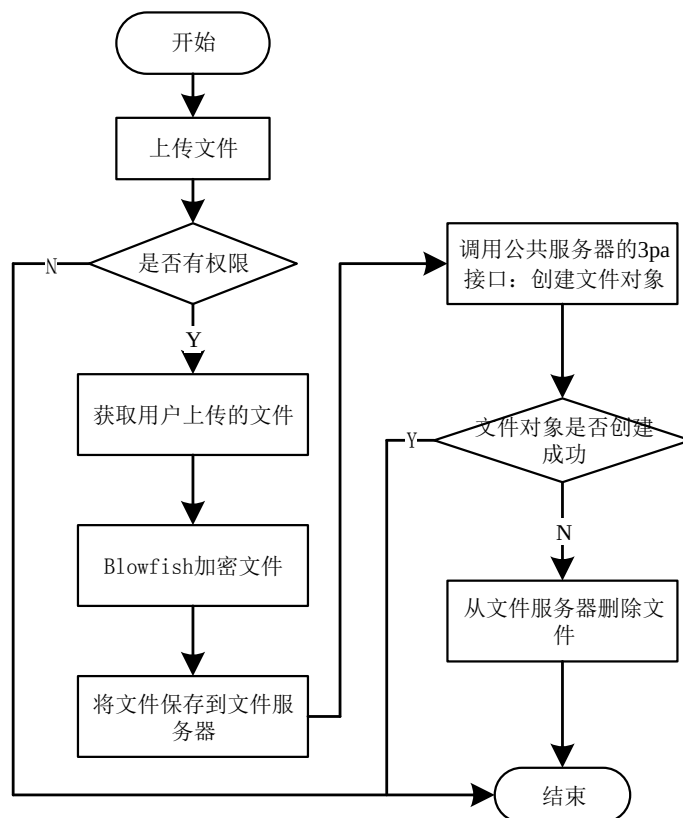


图 5-1 文件上传工具流程图

文件上传工具主要类如表 5-2 所示。

表 5-2 文件上传工具相关类

| 类名                    | 描述         |
|-----------------------|------------|
| FileUploadTool        | 工具的入口类     |
| DocumentDaoImpl       | 保存文件       |
| Document              | 文件属性       |
| HttpResult            | Http 返回结果  |
| UploadProcess         | 文件上传进度     |
| DocumentServiceImpl   | 和其它模块交互    |
| Constants             | 文件处理相关常量   |
| HttpClientUtil        | 封装 Http 请求 |
| UploadProcessListener | 文件上传进度监听器  |
| BaseDao               | 公共数据库访问    |

文件上传工具显示的字段有：添加版本、文件名、文件大小、上传进度信息、资料名称、关键字和描述信息。下面主要介绍文件上传的实现思路和过程。

用户点击 band.jsp 页面中资料柜的“上传”按钮，调出上传页面 documentPC.jsp，触发工具入口类 FileUploadTool 的 uploadDocument() 方法，在此方法中调用 UploadProcessListener 类中的 setProgressListener()方法，监听文件的上传进度。当用户填写完文件的基本信息：资料名称、关键字和描述信息之后，点击“提交”按钮，会调用 FileUploadTool 中的 confirmUploadDocument()方法，此方法调用 DocumentServiceImpl 类中的 getOrgIDByBandID()和 confirmUploadDocument()方法，分别获取用户所在的机构和在调用 BaseDao 类的 doPost()方法在公共服务器创建资料的记录，最后调用 convert()方法，请求负载均衡服务器。文件上传的时序图如图 5-2 所示。

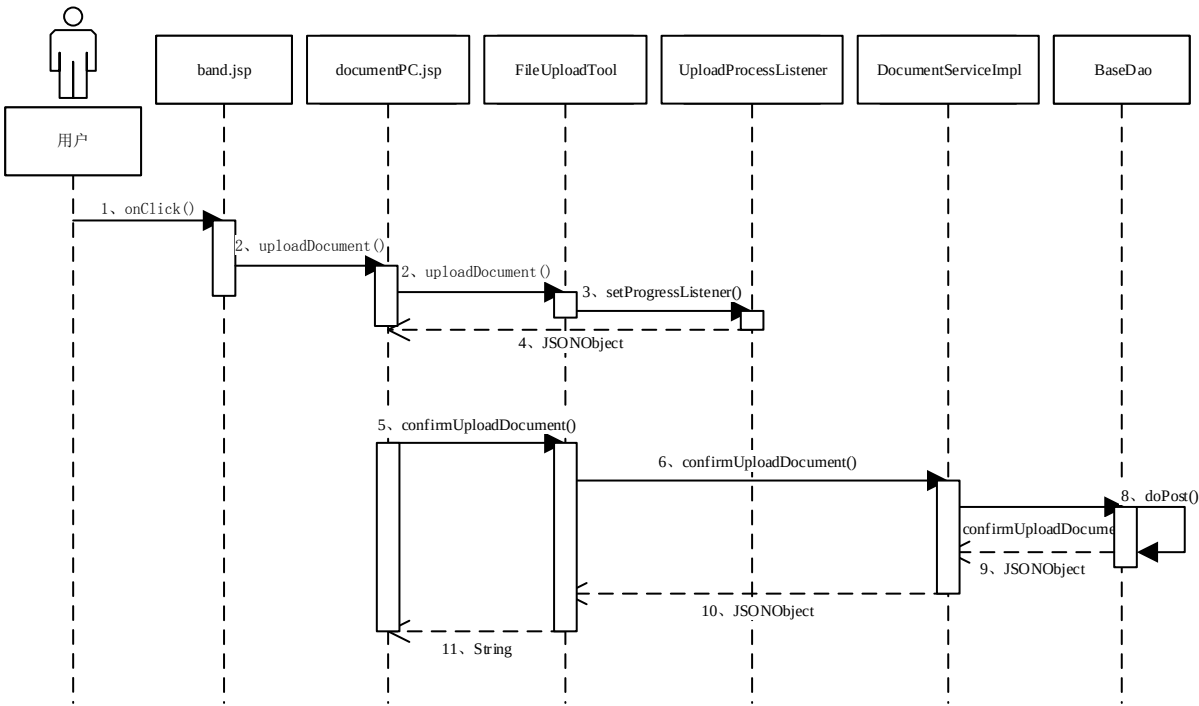


图 5-2 文件上传时序图

文件上传工具的整体类图如图 5-3 所示，从图中可以看出类之间的关系和调用的顺序。FileUploadTool 类继承实验室云平台提供的 Tool 类，是整个工具的入口类，其它类为其提供必要的服务。FileUploadTool 类中的 4 个方法依次用于上传文件、获取上传进度、重置上传文件和确认文件上传。

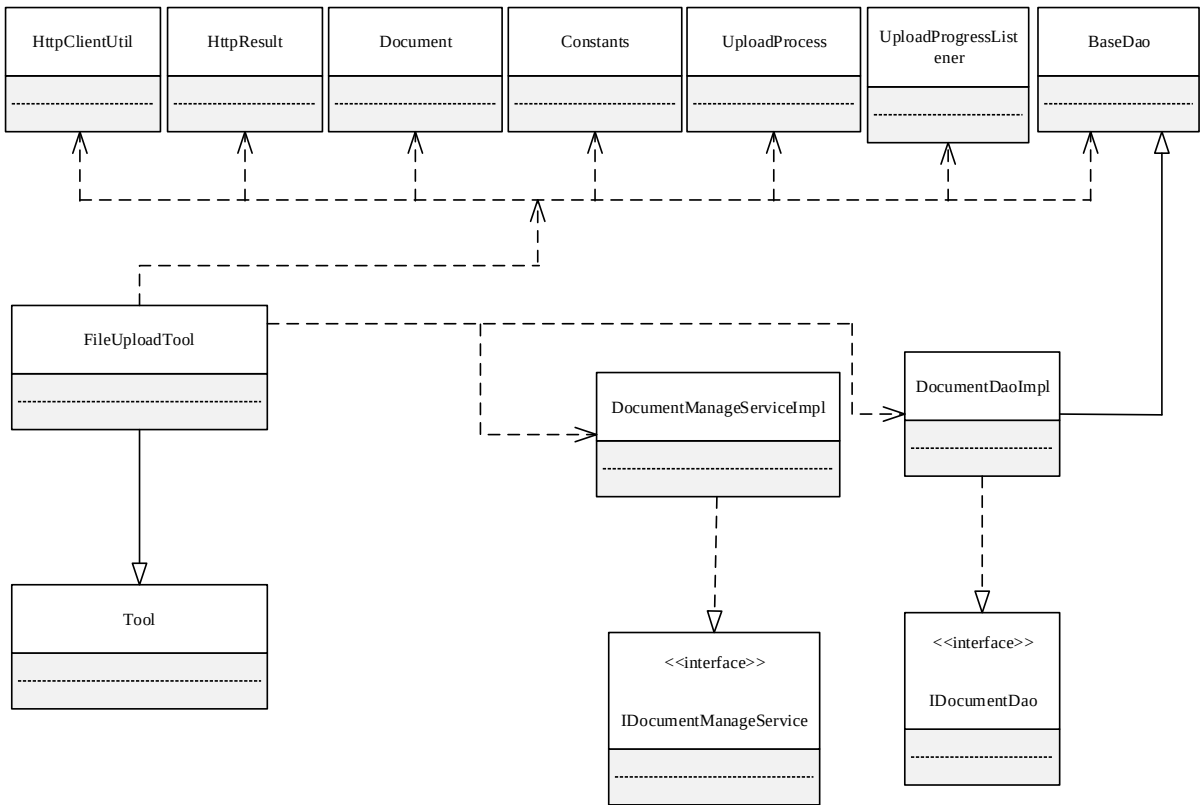


图 5-3 文件上传工具整体类图

其中部分类的属性和方法如图 5-4 所示。

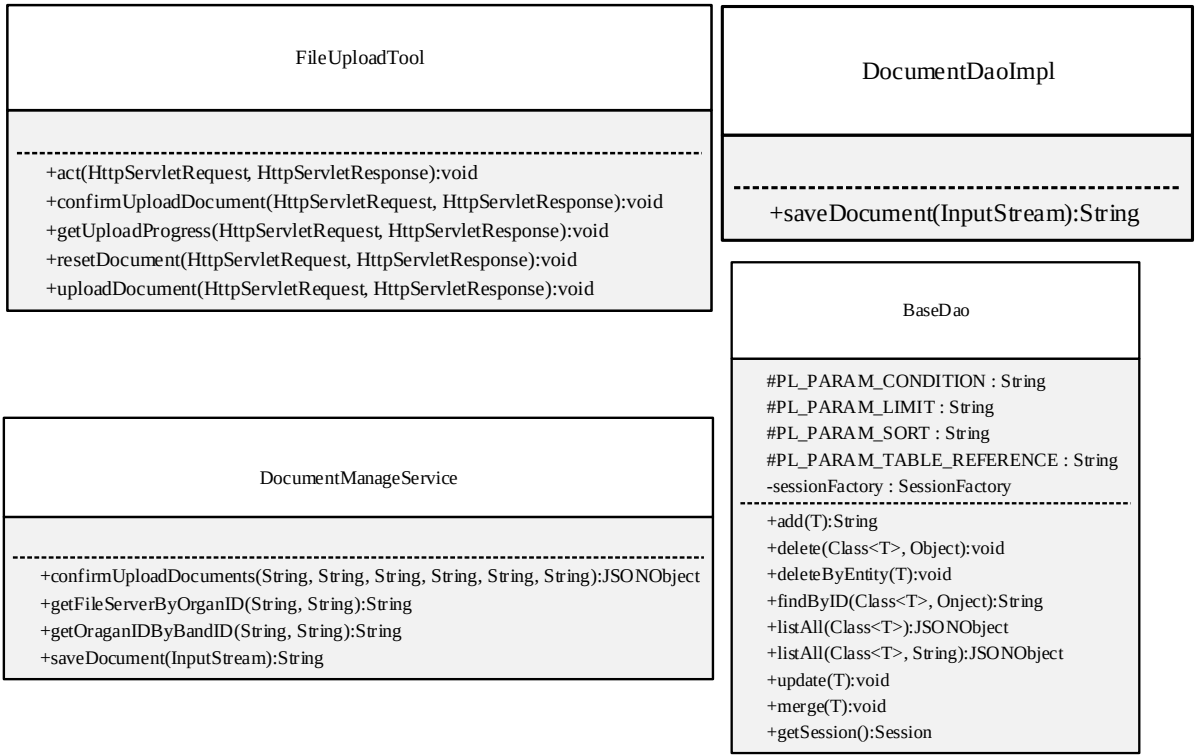


图 5-4 文件上传工具部分类的属性和方法

文件上传工具总共设计了 14 个类来实现文件上传业务的功能，下面介绍几个重要的方法和方法的用途，如表 5-3 所示。

表 5-3 文件上传工具重要的方法

| 方法名称                    | 描述       |
|-------------------------|----------|
| uploadDocument()        | 上传文件     |
| getUploadProgress()     | 获取文件上传进度 |
| confirmUploadDocument() | 确认文件上传   |
| resetDocument()         | 重置文件     |

5.2.2 文件预览工具

文件预览工具的前端采用 JavaScript 和 css3 来实现，后端采用 Java 实现。主要功能是完成文件在线预览和压缩包在线预览，文件预览工具的主要流程为：

- 1) 用户点击“预览”按钮。
- 2) 判断用户有无预览权限：如果有，则到第 3 步；如果无，则结束。
- 3) 根据文件的 storageID 在文件服务器的缓存区查找文件：如果存在缓存，则直接返还给用户；如果不存在缓存，则到第 4 步。
- 4) 根据请求文件的 storageID 从存储模块中读取相应的文件。
- 5) 使用 Blowfish 算法在文件服务器解密文件，并缓存文件。
- 6) 在文件预览工具节点生成相应的文件。
- 7) 根据文件后缀名解析：如果是压缩文件，则获取压缩文件文件夹的结构；如果是纯文本文件，则将纯文件以 GBK 编码；如果是其它格式，则不操作。
- 8) 返回相应的预览界面。

文件预览工具的流程图如图 5-5 所示。

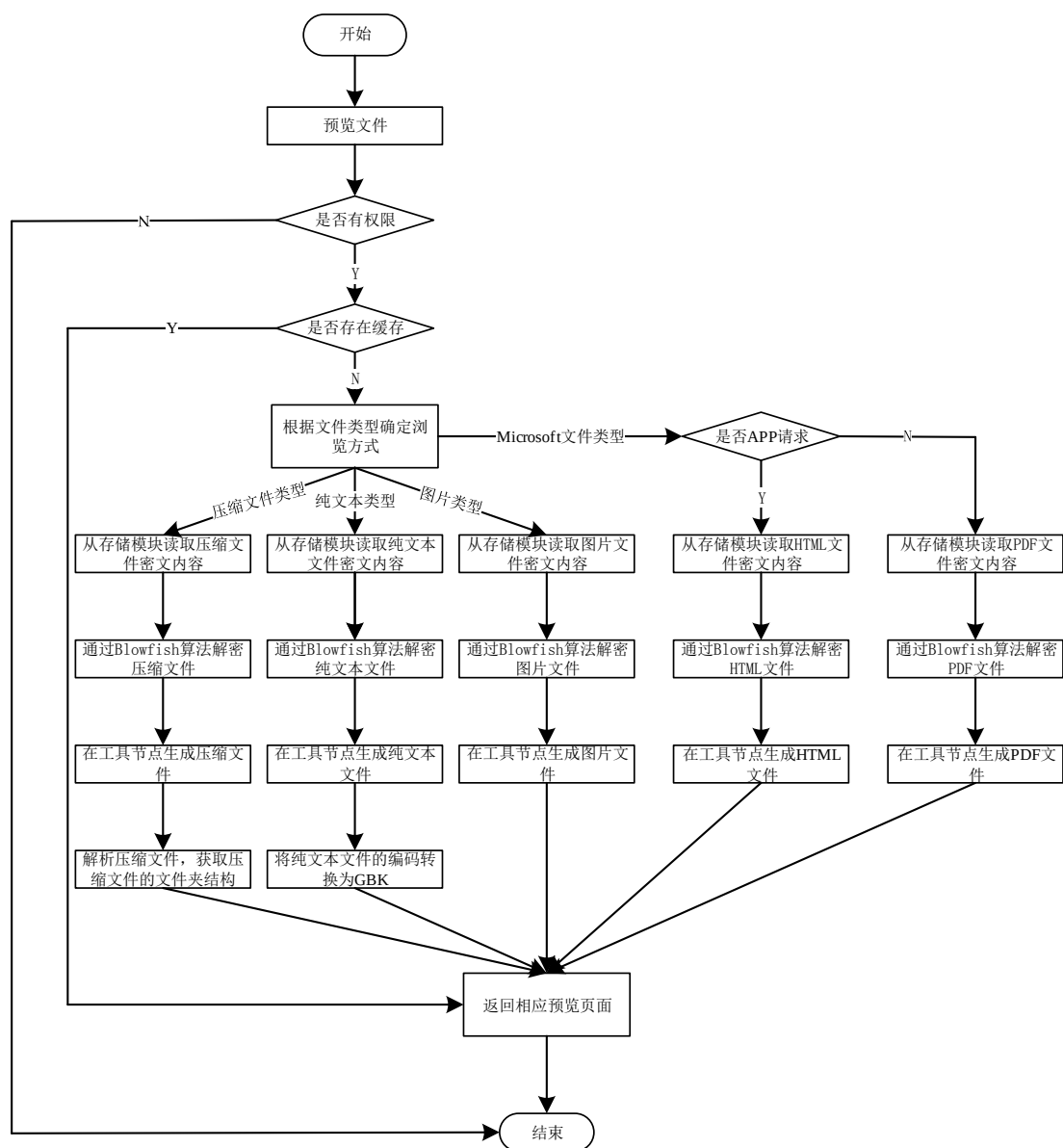


图 5-5 文件预览工具流程图

文件预览工具相关类如表 5-4 所示。

表 5-4 文件预览工具相关类

| 类名                     | 描述         |
|------------------------|------------|
| FilePreviewTool        | 工具的入口类     |
| FilePreviewImpl        | 操作缓存中的文件   |
| HttpResult             | Http 返回结果  |
| HttpClientUtil         | 封装 Http 请求 |
| FilePreviewServiceImpl | 文件预览业务实现   |



表 5-4 文件预览工具相关类（续）

|                |         |
|----------------|---------|
| DeCompressUtil | 解压压缩文件  |
| BaseDao        | 公共数据库访问 |

文件预览工具主要通过 PDF.js 插件预览文件，PDF.js 是由 HTML5 建立的 PDF 阅读器，依托开源社区的驱动和 Mozilla 实验室技术的支持，对浏览器的兼容性好，所以文件预览工具使用 PDF.js 预览文件。下面主要介绍文件预览工具的实现思路 and 过程。

用户点击 band.jsp 页面中资料柜的“预览”按钮，触发工具入口类 FilePreviewTool 的 act()方法，此方法中根据 request.getParameter(“toolAction”)得到预览文件的类型，从而调用不同的方法(如 filePreview())。在 filePreview ()中调用类 FilePreviewImpl 的 getCacheDocument()方法，从内存中获取文件；如果内存中没有，则根据文件的后缀名(如.rar)调用类 FilePreviewServiceImpl 中的 getFileContent()方法从存储模块中获取源文件，然后调用 getRARFile()方法，然后调用 DeCompressUtil 类中的 unrar()方法解压文件和 GetFilesPath()方法获得文件路径，最后返回预览文件页面。文件预览的时序图如图 5-6 所示。

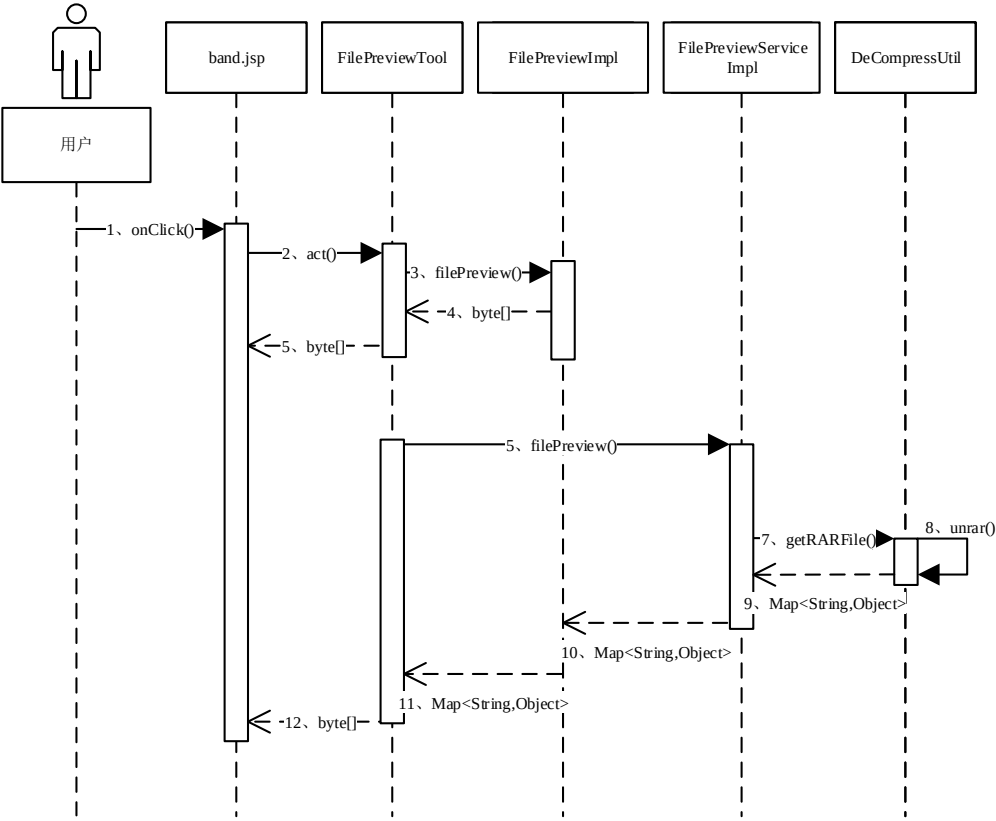


图 5-6 文件预览时序图

文件预览工具的整体类图如图 5-7 所示，从图中可以看出类之间的关系和调用的顺

序。FilePreviewTool 类继承实验室云平台提供的 Tool 类，是整个工具的入口类，其它类为其提供必要的服务。其中 FilePreviewImpl 类用于从缓存中获取数据，FilePreviewServiceImpl 类依赖 DeCompressUtil 类来实现文件预览的基本功能。FilePreviewTool 类中的 5 个方法依次用于预览压缩文件、预览 PDF 文件、预览 HTML 文件、预览图片和预览纯文本文件。

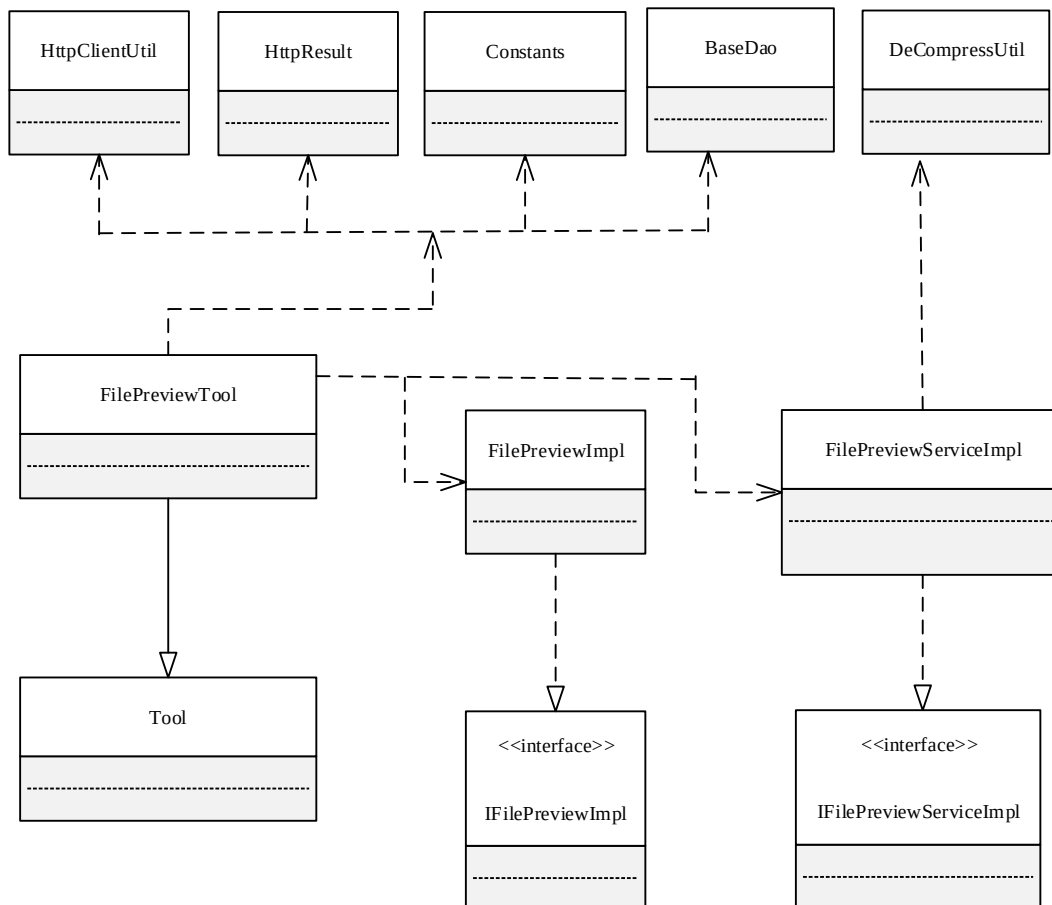


图 5-7 文件预览工具整体类图

其中部分类的属性和方法如图 5-8 所示，其中前面介绍到的公共类不再表述。

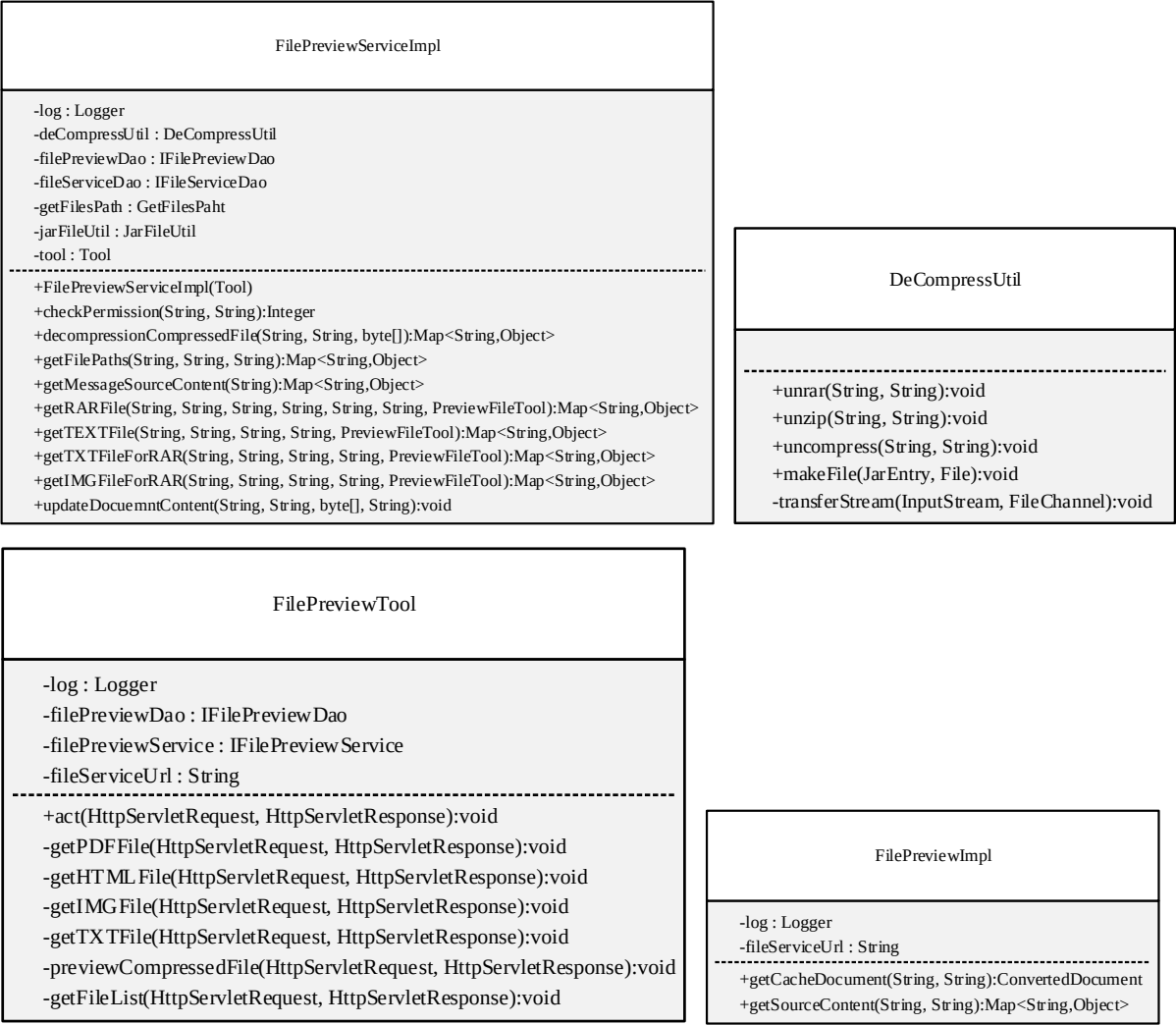


图 5-8 文件预览工具部分类的属性和方法

文件预览工具总共设计了 13 个类来实现文件预览业务的功能，下面介绍几个重要的方法和方法的用途，如表 5-5 所示。

表 5-5 文件预览工具重要的方法

| 方法名称                    | 描述         |
|-------------------------|------------|
| previewFile()           | 预览文件       |
| getFileList()           | 获取压缩文件里的文件 |
| previewCompressedFile() | 预览压缩文件     |
| getPDFFile()            | 获得 PDF 文件  |
| getHTMLFile()           | 获得 HTML 文件 |
| getIMGFile()            | 获得图片文件     |
| getTXTFile()            | 获得纯文本文件    |

### 5.2.3 文件下载工具

文件下载工具采用 Java 实现。主要功能是完成源文件的下载，文件下载工具的主要流程为：

- 1) 用户点击“下载”按钮。
  - 2) 判断用户有无预览权限：如果有，则到第 3 步；如果无，则结束。
  - 3) 根据文件的 storageID 在文件服务器的缓存区查找文件：如果存在缓存，则直接下载文件；如果不存在缓存，则到第 4 步。
  - 4) 根据文件的 storageID 请求存储模块，下载源文件，同时在文件服务器创建缓存。
- 文件下载工具的流程图如图 5-9 所示。

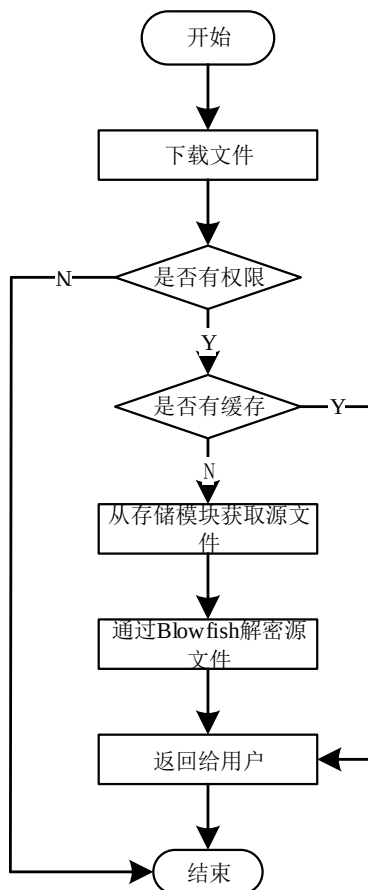


图 5-9 文件下载工具流程图

文件下载工具相关类如表 5-6 所示。

表 5-6 文件下载工具相关类

| 类名               | 描述     |
|------------------|--------|
| FileDownloadTool | 工具的入口类 |

表 5-6 文件下载工具相关类（续）

|                         |         |
|-------------------------|---------|
| FileDownloadServiceImpl | 获取文件    |
| Document                | 文件相关属性  |
| Constants               | 相关常量    |
| BaseDao                 | 公共数据库访问 |

文件下载工具的整体类图如图 5-10 所示，从图中可以看出类之间的关系和调用的顺序。FileDownloadTool 类继承实验室云平台提供的 Tool 类，是整个工具的入口类，其它类为其提供必要的服务。其中 FileDownloadServiceImpl 类依赖 Basedao 从存储模块中获取源文件或者从文件服务器中获得缓存源文件。FileDownloadTool 类中只有 downloadDocument()方法来实现在源文件下载。

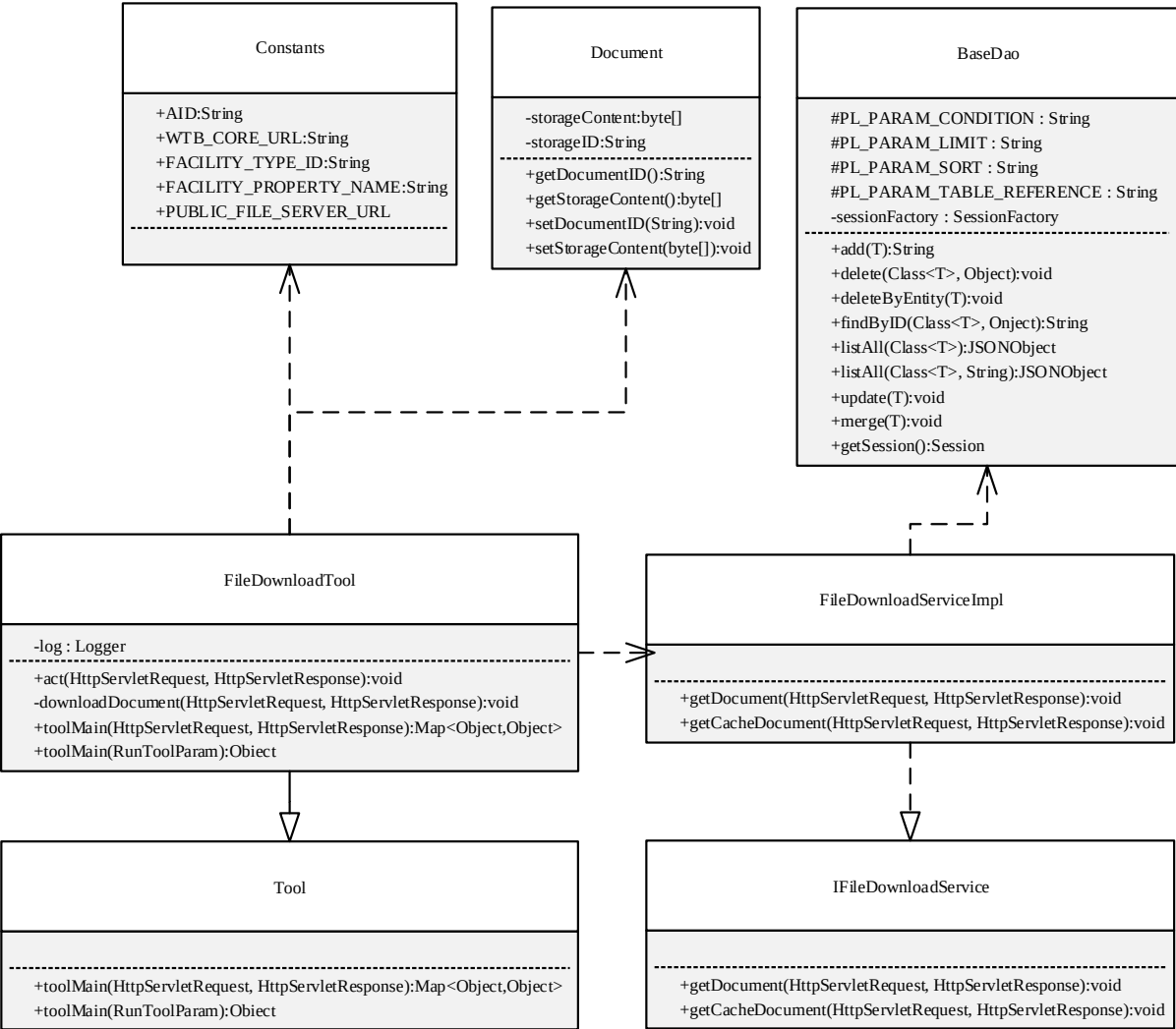


图 5-10 文件下载工具类图

文件预览工具总共设计了 7 个类来实现文件下载业务的功能，下面介绍几个重要的方法和方法的用途，如表 5-7 所示。

表 5-7 文件预览工具重要的方法

| 方法名称               | 描述        |
|--------------------|-----------|
| downloadDocument() | 下载文件      |
| getDocument()      | 存储模块获取源文件 |
| getCacheDocument() | 缓存获取源文件   |

#### 5.2.4 文件转换和切分工具

文件转换和切分工具采用 Java 实现，主要功能是对源文件的处理。在第三章中对文件转换和切分工具的需求和流程做了介绍，本小节主要针对文件转换和切分工具的具体实现。

文件转换和切分工具相关类如表 5-8 所示。

表 5-8 文件转换和切分工具相关类

| 类名                  | 描述              |
|---------------------|-----------------|
| FileManageTool      | 工具的入口类          |
| FileConvertServImpl | 文件转换服务          |
| FileSplitServImpl   | 文件切分服务          |
| Pdf2HtmlThread      | PDF 文件转 HTML 文件 |
| FileConvert         | 提供具体文件转换方法      |
| FileManageUtil      | 文件操作工具类         |
| DeCompressUtil      | 文件解压            |
| Document            | 文件实体            |
| Constants           | 公共常量            |
| FileConstants       | 文件相关常量          |
| StringValidate      | 验证参数是否有效        |

文件转换和切分工具的整体类图如图 5-11 所示，从图中可以看出类之间的关系和调用的顺序。FileManageTool 类继承实验室云平台提供的 Tool 类，是整个工具的入口类，其它类为其提供必要的服务。其中 FileConvertServImpl 类用于转换文件为 PDF 格式，FileConvert 提供具体的转换方法，在转换好的 PDF 文件通过 Pdf2HtmlThread 类转换为 HTML 格式文件；FileSplitServImpl 类主要将 PDF 格式的文件按大纲进行分割，分割的具体方法在下面介绍。FilePreviewTool 类中的方法，主要是进行文件转换。

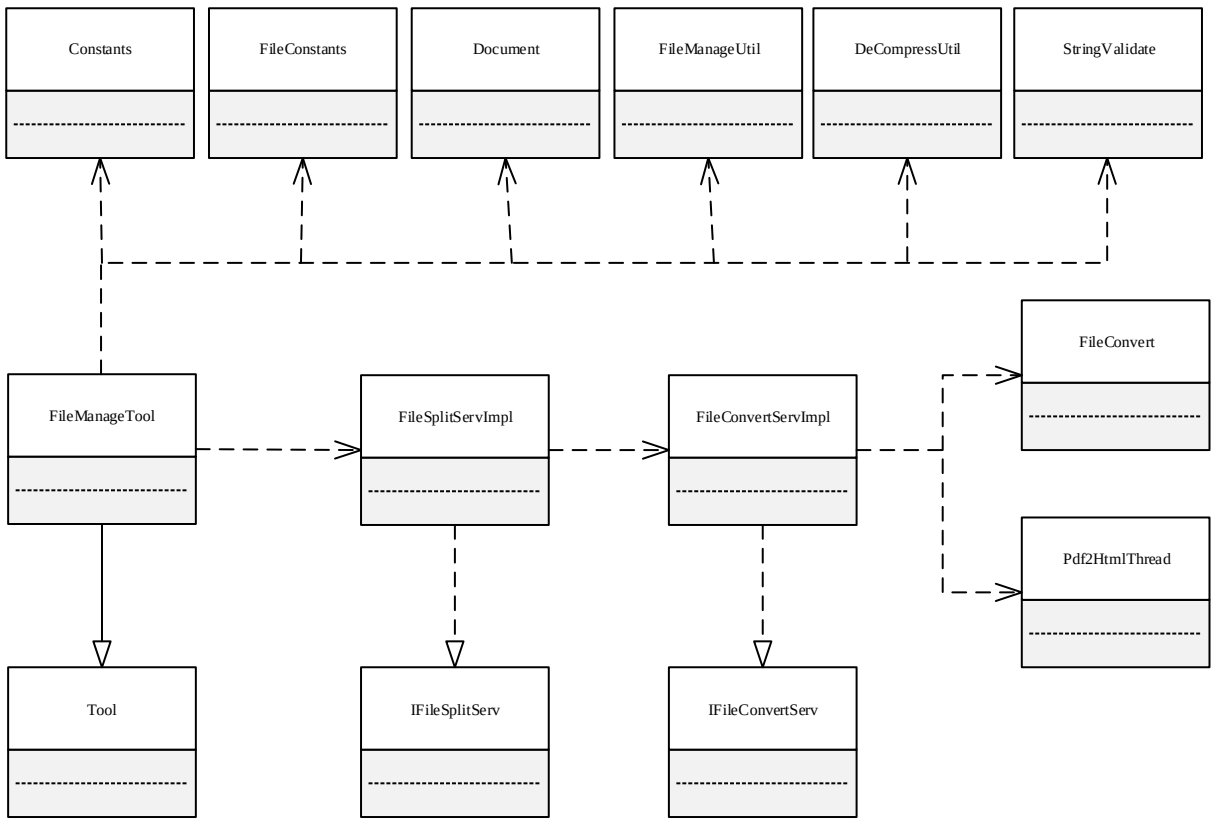


图 5-11 文件转换和切分工具的整体类图

其中部分类的属性和方法如图 5-12 所示，其中前面介绍到的公共类不再表述。



| FileConvertServiceImpl                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | FileConvert                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> -log : Logger -fileConverter : FileConverter -convertedCache : Map&lt;String, Map&lt;String, Object&gt;&gt; ----- +clearAllConvertedCache():void +convertAndInsertRARFile(String, String, byte[], String):Map&lt;String,Object&gt; +createSourceFile(String, String, byte[]):String +decompressionCompressedFile(String, String, byte[]):Map&lt;String,Object&gt; +getCoverInfo(HttpServletRequest, HttpServletResponse):void +msToPdf(String, String, String):boolean +pdfToHtml(String, String, String, String):boolean +saveFile(String, List&lt;String&gt;, List&lt;String&gt;, List&lt;String&gt;, List&lt;String&gt;):boolean +setConvertedCacheInfo(ConvertedDocument):void +setConvertedCacheInfo(String, Map&lt;String, Object&gt;):void </pre> | <pre> -log : Logger -EXCEL_HTML : int -EXCEL_PDF : int -fileConvertUtil : FileConvertUtil -PPT_PDF : int -VSD_PDF : int -WORD_PDF : int ----- +DOC2PDF(String, String, String):String +EXCELToPDF(String, String, String):String +PPTToPDF(String, String, String):String +VSDToPDF(String, String, String):String +PDF2Html(String, String, String):String </pre> |

图 5-12 文件转换和切分工具部分类的属性和方法

根据第二章 2.1 小节 PDF 文件格式的介绍，因为 PDF 文件的各个部分都由基本数据类型对象构成，所以可以根据 PDF 的逻辑结构来寻找文件的大纲。PDF 文件解析的入口是文件尾的 trailer 对象，根据 trailer 提供的根对象(/Root)号，找到大纲(outlines)位置，然后根据/First 和/Next 等属性，遍历 PDF 文件大纲章节的标题和页码，最后根据章节的页码将 PDF 文件分割为一个个的小章节。PDF 文件的字节流如图 5-13 所示。解析步骤如下：

#### 1) 查找文件尾 trailer

PDF 文件的尾部包含了根对象(/Root)的对象号和交叉引用表的位置，从而可以快速定位到 PDF 文件中的对象，因此 PDF 文件的解析一般从尾部开始，PDF 文件结束的标志是%%EOF。

#### 2) 定位根对象 Root

在文件尾 trailer 的属性标签/Root 中确定根对象的对象号，从而获得 PDF 文件逻辑结构的根节点。

#### 3) 定位大纲 Outlines

在 Root 的属性标签/Outlines 中确定大纲的对象号，从而得到大纲的位置。

#### 4) 定位各个章节的位置

根据大纲中的/First 和/Last 标签得到 PDF 文件第一个章节和最后一个章节的位置。例如：根据/First 标签得到了第一章(Chapter 1)，在 Chapter 1 中根据/Title 标签得到章节标题，根据/A 标签得到章节页码，根据/Next 标签得到相邻的章节 Chapter 2。在 Chapter 1 中的/First 标签得到子章节(Subsection)，子章节的/Next 标签得到相邻的子章节。依次



遍历到 PDF 文件的大纲。

```

34938 17832 0 obj
34939 <</MarkInfo<</Marked true>>/Names 17833 0 R/ViewerPreferences<</Direction/L2R>>/Outlines 2541 0 R/M
34940 endobj

34086 2541 0 obj
34087 <</First 17851 0 R/Count 21/Last 2575 0 R/Type/Outlines>>
34088 endobj

39393 17851 0 obj
39394 <</Parent 2541 0 R/A 17850 0 R/Next 17853 0 R/Title(Cover)>>
39395 endobj

34215 2617 0 obj
34216 <</First 2619 0 R/Parent 2541 0 R/A 2618 0 R/Next 2613 0 R/Count -9/Last 2620 0 R
34217 /Prev 2621 0 R/Title(Chapter 1: Getting Started with TensorFlow)>>
34218 endobj
34219 2618 0 obj
34220 <</D(p19)/S/GoTo/NewWindow true>>
34221 endobj

34221 2619 0 obj
34222 <</Parent 2617 0 R/A 2638 0 R/Next 2636 0 R/Title(?NULINULnNULcNULrNULoNULdNULuNULcNULtN
34223 endobj

34272 2636 0 obj
34273 <</Parent 2617 0 R/A 2637 0 R/Next 2634 0 R/Prev 2619 0 R/Title(?NULHNULcNULwNUL NULtNU
34274 endobj
34275 2637 0 obj
34276 <</D(p20)/S/GoTo/NewWindow true>>

57463 trailer
57464 <</Size 17950/Root 17832 0 R>>
57465 xref
57466 0 0
57467 trailer
57468 <</Size 17950/Prev 4407226/XRefStm 284078/Root 17832 0 R/Info 3008 0 R/ID[<C1283BDE0F609C4790C
57469 startxref
57470 4766282
57471 %%EOF

```

图 5-13 PDF 文件字节流

寻找大纲章节的流程如图 5-14 所示

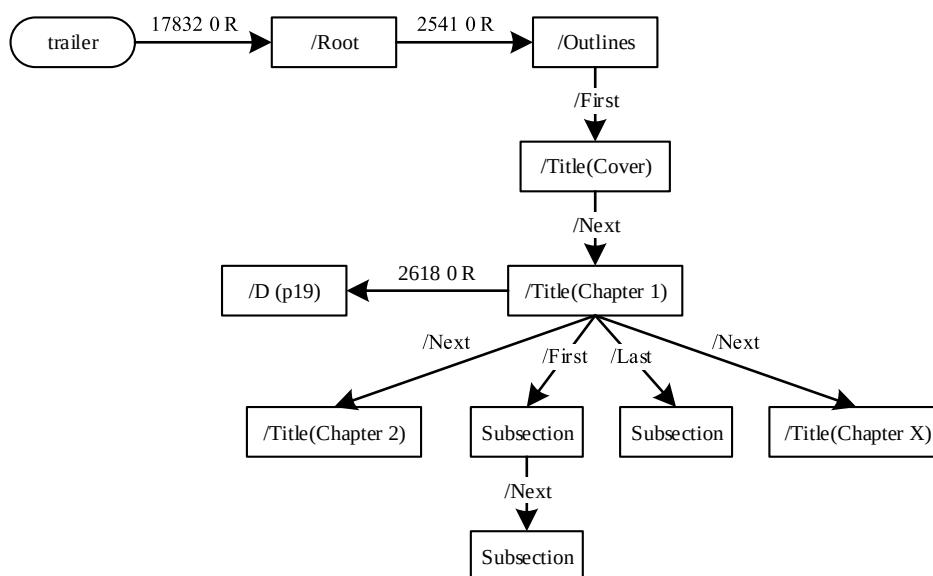


图 5-14 PDF 文件大纲解析流程

在获得大纲中各个章节的标题和页码之后，调用 `splitPdf()` 方法对 PDF 文件按章节切分，获得章节和切分的流程图如图 5-15 所示。

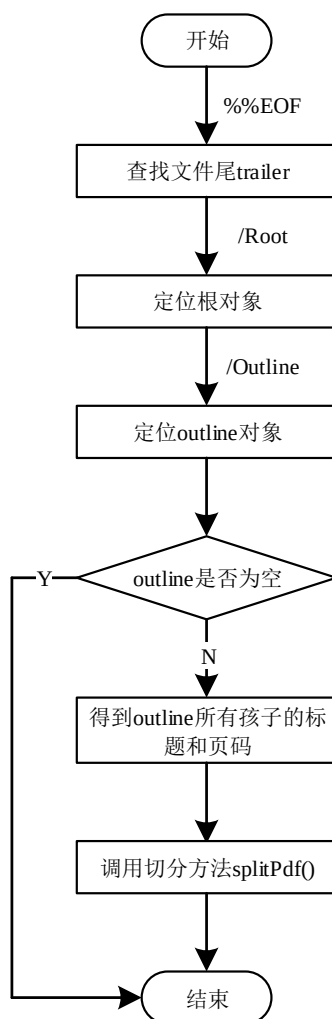


图 5-15 PDF 的切分流程

文件转换和切分工具总共设计了 28 个类来实现文件转换和切分业务的功能，下面介绍几个重要的方法和方法的用途，如表 5-9 所示。

表 5-9 文件转换和切分工具重要的方法

| 方法名称                              | 描述                    |
|-----------------------------------|-----------------------|
| <code>convertFile()</code>        | 转换文件                  |
| <code>getConvertInfo()</code>     | 获取文件转换信息              |
| <code>clearConvertedFile()</code> | 清除转换的文件               |
| <code>isBandFile()</code>         | 判断文件是否为帮区文件           |
| <code>msToPdf()</code>            | Microsoft 格式文件转换为 PDF |
| <code>pdfToHtml()</code>          | PDF 格式转换为 HTML        |

表 5-9 文件转换和切分工具重要的方法（续）

|               |             |
|---------------|-------------|
| getOutlines() | 获取 PDF 文件大纲 |
| splitPdf()    | 切分 PDF 文件   |

### 5.3 工具展示

本节主要展示 5.2 小节所实现的文件上传工具、文件预览工具和文件下载工具，文件转换和切分工具为无页面工具故不展示。

图 5-16 为文件上传和下载工具。

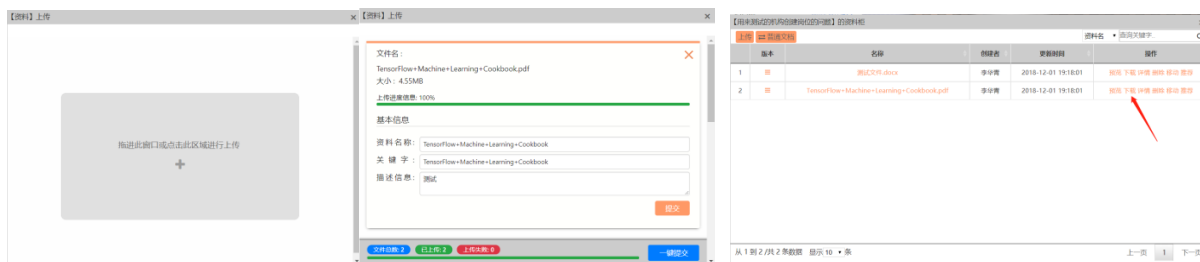


图 5-16 文件上传和下载工具

图 5-17 为使用文件预览工具预览单个文件，点击“预览”按钮时，如果文件存在目录，则返回给用户文件的目录，然后用户选择具体的章节，文件在线预览服务再将具体的章节返回给用户；如果文件没有目录，则直接将完整的文件返回给用户。图 5-18 为使用文件预览工具预览压缩文件。

#### Contents

|                                             |      |
|---------------------------------------------|------|
| Website                                     | vii  |
| Acknowledgments                             | viii |
| Notation                                    | xi   |
| 1 Introduction                              | 1    |
| 1.1 Who Should Read This Book?              | 8    |
| 1.2 Historical Trends in Deep Learning      | 11   |
| 1 Applied Math and Machine Learning Basics  | 29   |
| 2 Linear Algebra                            | 31   |
| 2.1 Scalars, Vectors, Matrices and Tensors  | 31   |
| 2.2 Multiplying Matrices and Vectors        | 34   |
| 2.3 Identity and Inverse Matrices           | 36   |
| 2.4 Linear Dependence and Span              | 39   |
| 2.5 Norms                                   | 40   |
| 2.6 Special Kinds of Matrices and Vectors   | 42   |
| 2.7 Eigendecomposition                      | 44   |
| 2.8 Singular Value Decomposition            | 45   |
| 2.9 The Moore-Penrose Pseudoinverse         | 46   |
| 2.10 The Trace Operator                     | 47   |
| 2.11 The Determinant                        | 48   |
| 2.12 Example: Principal Components Analysis |      |

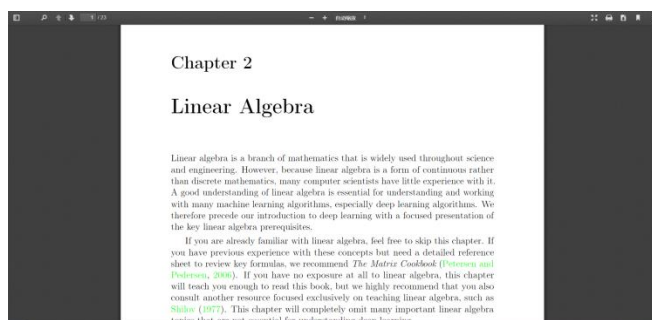


图 5-17 预览单个文件



图 5-18 预览压缩文件

## 5.4 本章小结

本章以实验室科研团队提出的“工具模型”为基础，设计并实现了文件在线预览服务相关的工具，并对相关工具进行展示，验证了文件在线预览服务所需的功能。在文件转换和切分工具中着重介绍了 PDF 文件的解析流程，从而获得 PDF 文件的章节。

## 第六章 文件在线预览服务实验分析

对文件在线预览服务进行测试，主要目的在于检验各个工具能否正常运行，以及在高负载的网络请求下，工具运行节点的负载能够相对均衡。

本文的实验包含两个部分：一个是文件在线预览服务的功能测试，验证文件在线预览服务是否实现了相应的功能；一个是文件在线预览服务的性能测试，验证文件在线预览服务在高并发的情况下快速响应用户的请求。

### 6.1 实验环境

本小节将具体介绍实验的环境，为后面的负载测试提供基础。实验环境包括设计的网络拓扑结构、服务器的选择、软件选择等工作。

#### 6.1.1 网络拓扑结构

实验环境的拓扑结构如图 6-1 所示。

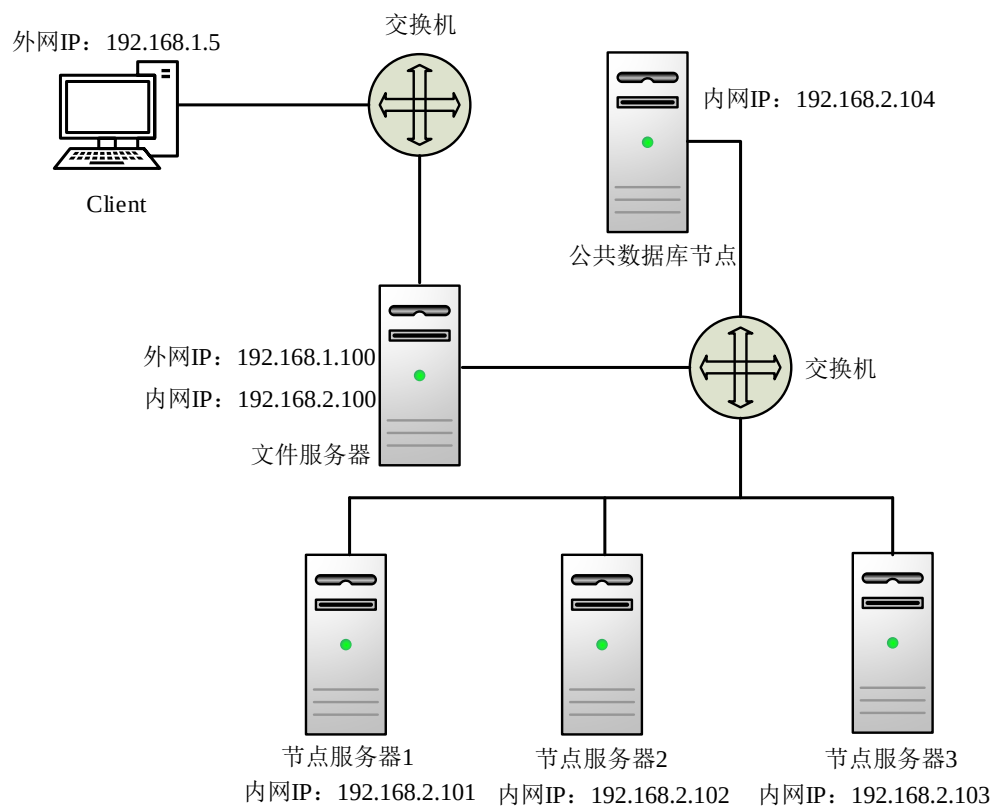


图 6-1 网络拓扑结构

客户端(Client)和文件服务器通过一个交换机连接。文件服务器、各个节点服务器和公共数据库节点通过一个交换机连接，两个交换机一个用于连接外网，一个用于连接内网。网络拓扑的网段信息如表 6-1 所示。

表 6-1 网段信息

| 网段名称 | 网段表示           |
|------|----------------|
| 外网   | 192.168.1.1/24 |
| 内网   | 192.168.2.1/24 |

通过图 5-18 可知 Client 将模拟用户操作, 运行文件上传工具、文件预览工具和文件下载工具; 因为文件上传工具、文件预览工具和文件下载工具功能简单, 占用系统资源少, 所以将这三个工具部署在文件服务器上, 同时文件服务器也是件转换和切分工具的调度中心, 用来分发文件转换请求; 3 个节点服务器用来运行文件转换和切分工具和部署 MongoDB 分片集群; 公共数据库节点存储系统的共享数据。在图中已经标出每台机器的 IP 地址, 所有的 IP 地址为静态 IP 地址。

### 6.1.2 实验环境硬件配置

实验使用到的各个机器的硬件配置如表 6-2 所示, 其中使用了 3 个服务器节点, 每个节点的硬件配置各不相同。

表 6-2 机器硬件配置

| 名称      | CPU 信息               | 内存信息 | 网卡信息      | 磁盘信息  |
|---------|----------------------|------|-----------|-------|
| Client  | core i5, 4 核, 3.2GHZ | 8GB  | 1 个千兆以太网卡 | 1TB   |
| 文件服务器   | core i5, 4 核, 3.2GHZ | 8GB  | 2 个千兆以太网卡 | 1TB   |
| 节点服务器 1 | core i5, 4 核, 3.2GHZ | 8GB  | 1 个千兆以太网卡 | 1TB   |
| 节点服务器 2 | core i3, 2 核, 3.3GHZ | 4GB  | 1 个千兆以太网卡 | 500GB |
| 节点服务器 3 | core i3, 2 核, 2.8GHZ | 4GB  | 1 个千兆以太网卡 | 320GB |
| 公共数据库节点 | core i5, 4 核, 3.2GHZ | 8GB  | 1 个千兆以太网卡 | 500GB |

### 6.1.3 实验环境软件配置

在 Client 上安装 LoadRunner 模拟高负载网络请求, 在文件服务器安装 Nginx 1.10.2, 在 3 个节点服务器部署 MongoDB 3.4, 在公共数据库节点安装 MySQL 5.7。各个机器的软件配置如表 6-3 所示。

表 6-3 机器软件配置

| 名称     | 操作系统          | 应用软件                           |
|--------|---------------|--------------------------------|
| Client | CentOS 7 64 位 | autobench 2.1.2                |
| 文件服务器  | CentOS 7 64 位 | Nginx 1.10.2、Java 1.8、Tomcat 8 |

表 6-3 机器软件配置（续）

|         |               |                               |
|---------|---------------|-------------------------------|
| 节点服务器 1 | CentOS 7 64 位 | Java 1.8、Tomcat 8、MongoDB 3.4 |
| 节点服务器 2 | CentOS 7 64 位 | Java 1.8、Tomcat 8、MongoDB 3.4 |
| 节点服务器 3 | CentOS 7 64 位 | Java 1.8、Tomcat 8、MongoDB 3.4 |
| 公共数据库节点 | CentOS 7 64 位 | MySQL 5.7                     |

使用 3 台节点服务器来搭建 MongoDB 集群，主要采用分片加副本集的策略，其配置信息如表 6-4 所示。

表 6-4 集群节点配置

| 主机      | 服务             | 端口    |
|---------|----------------|-------|
| 节点服务器 1 | Shard1_Master  | 27001 |
|         | Shard2_Slave   | 27002 |
|         | Shard3_Arbiter | 27003 |
|         | Config         | 21000 |
|         | Mongos         | 20000 |
| 节点服务器 2 | Shard2_Master  | 27001 |
|         | Shard3_Slave   | 27002 |
|         | Shard1_Arbiter | 27003 |
|         | Config         | 21000 |
|         | Mongos         | 20000 |
| 节点服务器 3 | Shard3_Master  | 27001 |
|         | Shard1_Slave   | 27002 |
|         | Shard2_Arbiter | 27003 |
|         | Config         | 21000 |
|         | Mongos         | 20000 |

## 6.2 功能测试

本次实验测试文件在线预览服务的相关功能，其具体的测试用例和测试结果如表 6-5 所示。

表 6-5 文件在线预览服务功能测试

| 功能模块 | 用例说明    | 预期结果                      | 实际结果                     | 是否通过 |
|------|---------|---------------------------|--------------------------|------|
| 文件上传 | 上传文件    | 用户能够上传单文件、多个文件和更新文件版本     | 用户在资料柜上传单文件、多个文件更新文件版本   | 是    |
| 文件下载 | 下载文件    | 用户能够下载文件                  | 用户在资料柜下载文件               | 是    |
| 文件转换 | 转换文件格式  | 用户上传的源文件转换为 PDF 和 HTML 格式 | 文件上传后能够转换为 PDF 和 HTML 格式 | 是    |
| 文件切分 | 按章节切分文件 | 文件如果有章节,按章节切分文件           | 文件能够按章节切分                | 是    |
| 文件预览 | 文件显示    | 显示预览文件                    | 用户预览文件正常显示               | 是    |
|      | 文件翻页    | 支持上一页、下一页和页面跳转            | 用户预览文件时能够翻页              | 是    |
|      | 文件缩放    | 文件能够按比例缩放                 | 用户能够随意缩放文件               | 是    |
|      | 文件打印    | 直接打印文件                    | 用户能够打印文件                 | 是    |
|      | 文件加载    | 预览异常,可以重新加载文件             | 网络异常,重新加载文件              | 是    |
|      | 文件提示    | 预览异常,提醒用户                 | 出现异常情况,通知用户              | 是    |
| 日志   | 预览日志    | 记录系统的操作                   | 生成预览日志                   | 是    |
| 文件报错 | 通知管理员   | 预览文件出错,通知管理员              | 用户无法正常预览文件,通知管理员         | 是    |

### 6.3 集群性能测试

本次实验的性能测试主要在两个方面：一个是面对高并发的文件上传时，文件转换和切分工具的负载均衡；另一个是 MongoDB 数据库存储时对热点数据的均衡，满足高



并发的用户读写请求。所以实验内容为：

1) 方案 1：面对高并发文件上传时产生的文件转换和切分任务，本文基于 Nginx 的动态加权轮询算法能够均衡文件转换和切分工具服务器的负载，减少集群的响应时间。

2) 方案 2：MongoDB 能够根据数据访问的热度不同，将冷热数据均衡的分布在不同的分片服务器中，使得各个服务器能够负载均衡，减少文件的读写时间。

### 6.3.1 方案 1 测试和分析

由经验可知在高负载网络请求下，服务器集群的性能一般优于单个服务器的性能，因此本文不对集群和单个服务器的性能作比较；服务器的硬件配置越高，性能越好，也不比较实验中 3 个节点服务器的单点性能。

此方案比较 Nginx 的静态加权轮询算法和本文改进的实时更新节点服务器权重的动态反馈轮询算法。实验中的文件服务器可以根据系统中节点服务器的硬件信息计算出各个节点服务器的初始权重，根据公式(3.1)和表 6-2 中的数据，可以计算出节点服务器 1 的初始权重为 14，节点服务器 2 的初始权重为 9，节点服务器 3 的初始权重为 8，所以在实验刚开始时，三个节点服务器的初始权重分别设置为：14，9，8。

为了测试集群在不同负载下的性能，在实验的过程中对某个节点人为的添加消耗 CPU 运行时间和内存空间的程序，如图 6-2 所示。

```
int memConsumed[] = new int[150*1024*1024]; // 消耗600MB内存
int i = 0;
while(true){
    i++;
    if(i%10000 == 0){
        i = 0;
        try{
            Thread.sleep(1);
        }catch(InterruptedException e){
            e.printStackTrace();
        }
    }
}
```

图 6-2 模拟负载

#### 1) 静态加权轮询算法负载测试

文件服务器在对文件转换和切分工具进行负载均衡时，采用 Nginx 默认的静态加权轮询算法，测试结果如图 6-3 所示。

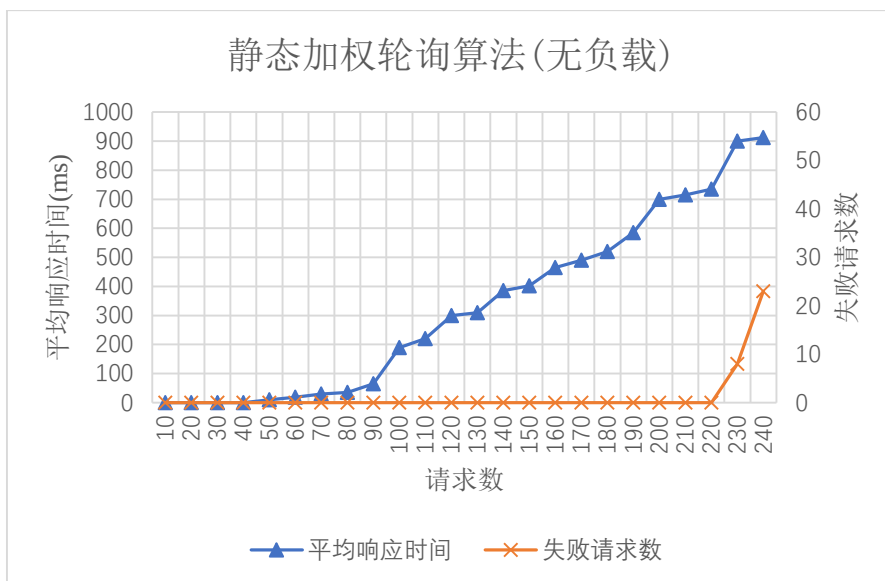


图 6-3 静态加权轮询算法测试结果

从图 6-3 中可以看出集群的处理能力有限，随着文件转换和切分的请求增加，工具的响应时间也逐渐增加，当请求数达到 220 左右的时候，开始出现失败的请求。出现失败请求后，在增加文件转换和切分的请求，失败的请求数也会线性增加。

在节点服务器 1 上添加模拟负载，再次对静态加权轮询算法进行负载测试，测试结果如图 6-4 所示。

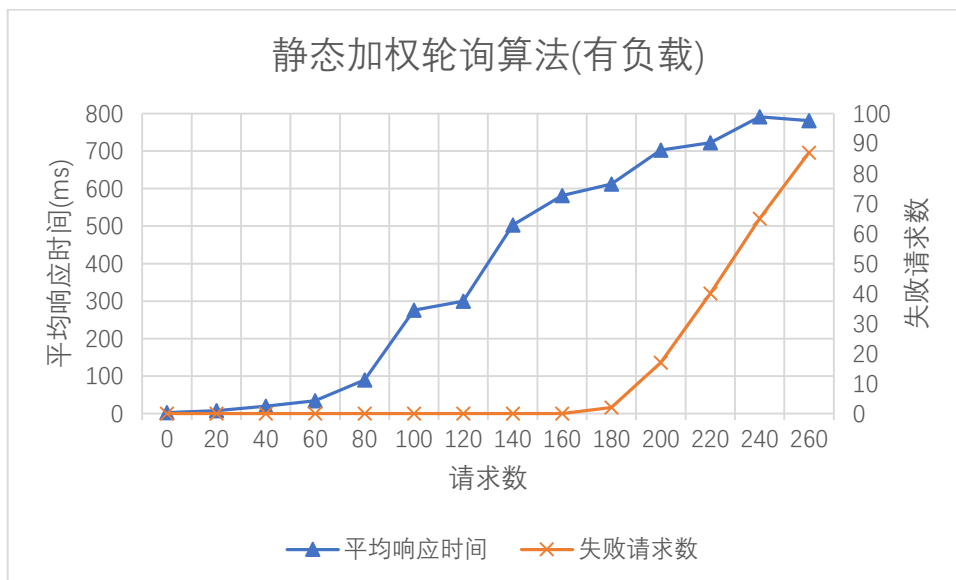


图 6-4 模拟负载，静态加权轮询算法测试结果

由图 6-3 和图 6-4 可知，集群在增加负载后，平均响应时间的增幅明显加快，而且集群并发数的阈值由未增加模拟负载的 220 左右降到了 180 左右，由此可知 Nginx 默认的静态加权轮询算法没有及时应对后端节点服务器的资源利用情况变化，未能根据节点服务器的负载情况合理分配文件转换和切分请求。

## 2) 动态加权轮询算法负载测试

在没有添加模拟负载的时候,动态加权轮询算法和静态加权轮询算法的测试结果类似,因此在添加模拟负载之后对动态加权轮询算法进行测试。具体的负载结果如图 6-5 所示。

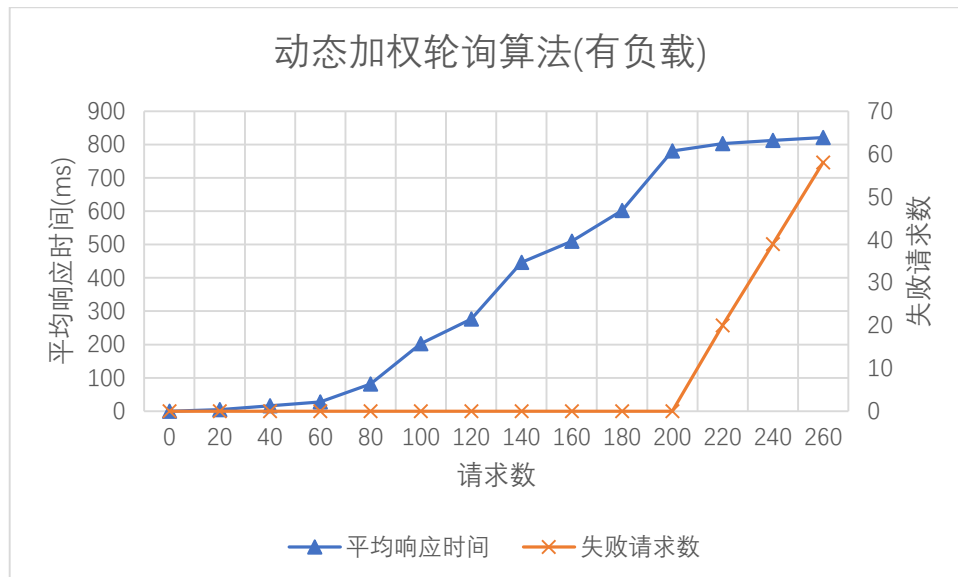


图 6-5 模拟负载，动态加权轮询算法测试结果

通过图 6-4 和图 6-5 对比可知,在有模拟负载的情况下,动态加权轮询算法比静态加权轮询算法有更低的响应时间和集群的并发数有所提升。

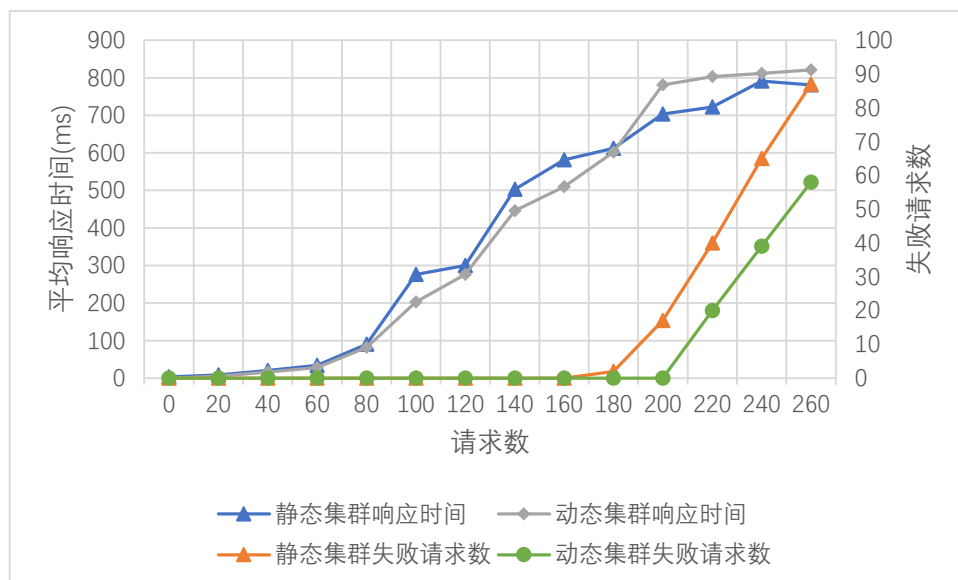


图 6-6 模拟负载，静态和动态加权轮询算法测试结果

通过图 6-6 可知,在有模拟负载的情况下,在正常的并发负载下,动态加权轮询算法的响应时间要低于静态加权轮询算法,只是在出现失败请求后响应时间发生变化;动态加权轮询算法比静态加权算法有更高的并发数,而且失败请求数的变化趋势要慢于静

态加权轮询算法。

综合方案 1 的实验分析，可以得出：本文研究的动态加权轮询算法与 Nginx 的静态加权轮询算法相比能够有效应对后端节点服务器不断变化的负载情况，调整节点服务器的权重，从而提高集群的整体性能，降低集群的响应时间。

### 6.3.2 方案 2 测试和分析

使用 Linux 下的 dd 命令随机生成 5000、10000、20000、50000 和 100000 个文件，由于文件在线预览服务需要将 PDF 文件按章节拆分、存储，所以 MongoDB 中存储的文件不会太大。因此，本文生成文件的大小分布为：20KB、50 KB、100 KB 和 500 KB 的文件总数占 80%；1MB、2 MB、10 MB 和 20 MB 的文件总数占 20%。

分别比较 MongoDB 自带的数据均衡策略和本文提出的基于冷热数据分布均衡的改进策略在连续写不同数量文件的用时时间；然后读取前 5000、10000、20000 和 50000 个文件，比较 2 种策略的用时时间。在测试文件读写性能的时候，每次做 5 次实验然后取平均值。

#### 1) 不存在“热数据”时读写文件用时对比

通过图 6-7 和图 6-8 可知，在读写文件用时上，本文提出的基于热负载均衡策略要比 MongoDB 原有的数据均衡策略用时时间稍高，因为此时数据还没有冷热之分，但是本文策略需要计算数据块和分片的热权重，以及数据块迁移判断，这些操作会给分片服务器增加一些负载，所以与原策略相比较用时时间稍高。

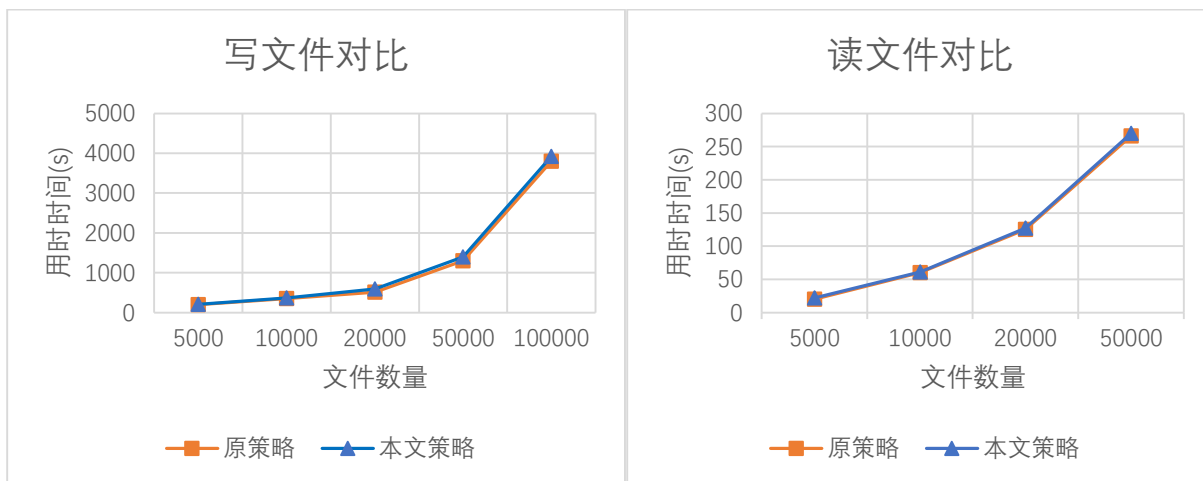


图 6-7 写文件时间对比

图 6-8 读文件时间对比

#### 2) 存在“热数据”时读写文件用时对比

通过图 6-9 和图 6-10 可知，在读写文件用时上，本文提出的基于热负载均衡策略要比 MongoDB 原有的数据均衡策略用时时间显著减少，因为本文策略会将“热数据”均

衡分布在各个分片服务器上，均衡集群服务器的负载，提示集群的性能。

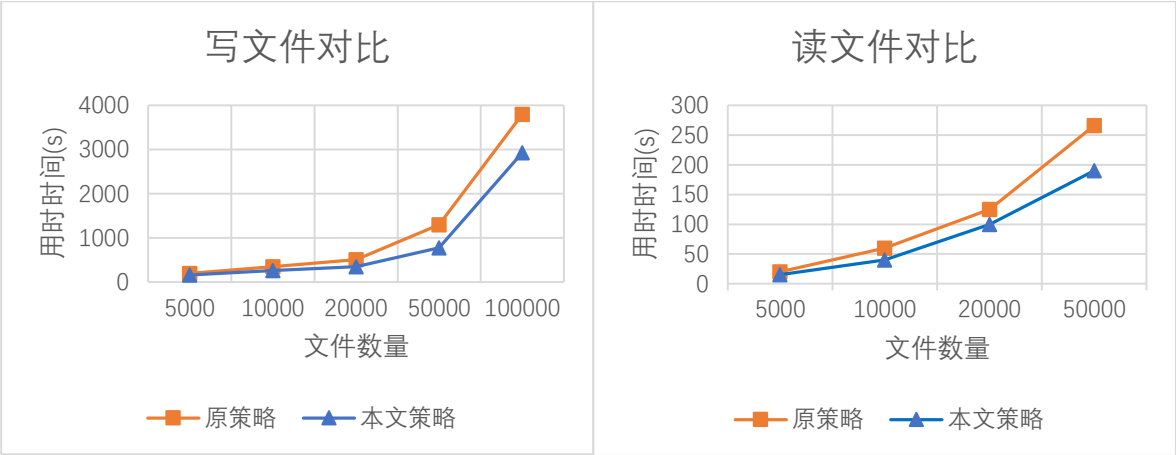


图 6-9 写文件时间对比

图 6-10 读文件时间对比

3) 存在“热数据”时节点服务器访问数对比

在读写文件实验做完后，关闭文件服务器的缓存功能，然后做 500 次随机读文件的实验(一共做 5 次实验)，记录每个节点服务器被访问的次数。

通过图 6-11 和图 6-12 对比可知，MongoDB 默认的数据均衡算法不能在节点服务器均衡“热数据”，这导致在每 500 次随机访问的时候，被访问的热点数据集中存储在单个节点服务器中，使得此节点服务器的负载压力大，而其它的节点服务器夫在压力小，容易出现“木桶效应”，影响整个集群的性能；而本文基于热负载均衡策略，能够将“热数据”均衡存储在各个节点服务器，这样每个节点服务器能够均分负载，使得集群的整体性能得到提高。

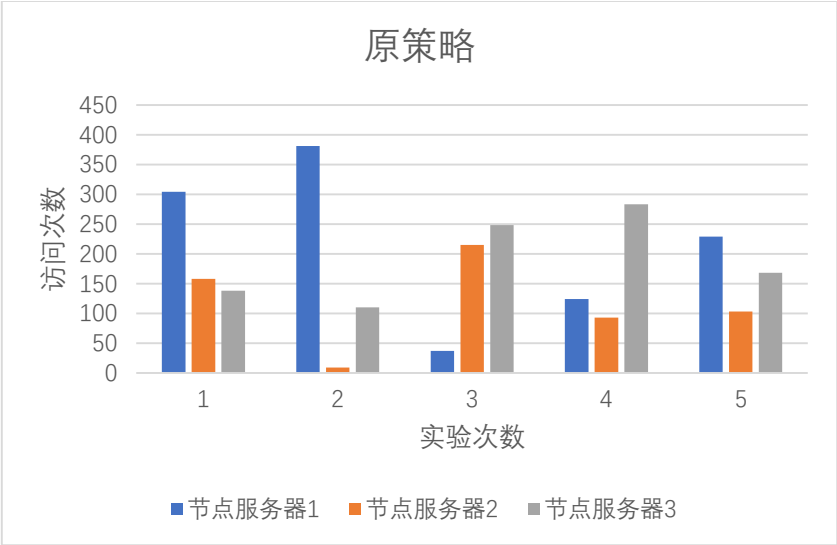


图 6-11 MongoDB 数据均衡

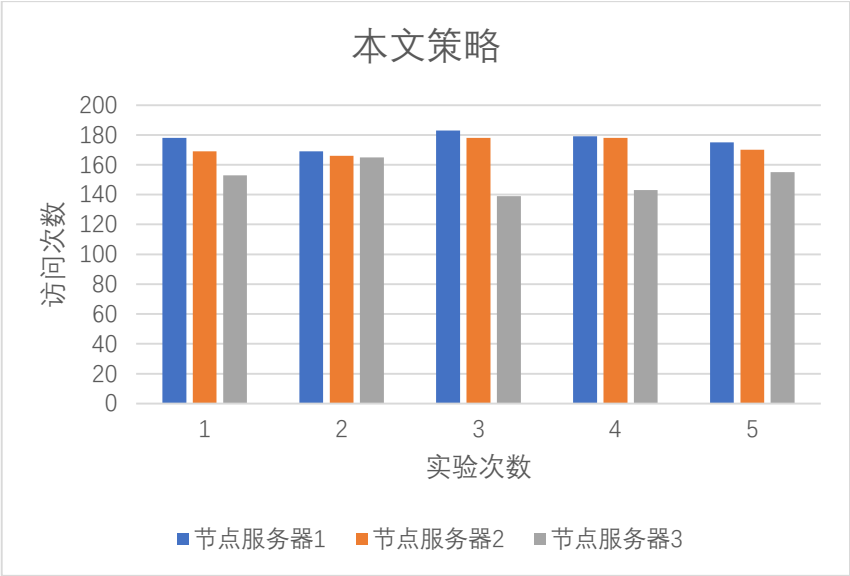


图 6-12 热负载均衡策略

综合方案 2 的实验分析，可以得出：本文研究的基于队列的热数据识别算法能够根据数据块的热权重和分片服务器的资源利用率迁移数据块，均分各分片节点的热负载，抑制“木桶效应”现象，从而提升集群的整体性能，降低集群的响应时间。

### 6.4 本章小结

本章对文件在线预览服务的功能和性能做了测试。本章的测试主要用来检验服务能否对负载进行合理调度，使得在高负载的网络请求下文件转换和切分工具的负载相对均衡，MongoDB 中热点数据的存储也能相对均衡。实验最终表明，文件在线预览服务的功能和性能均达到了预期的效果。

## 总结与展望

### 论文总结

本文在研究文件在线预览服务的过程中，摒弃了以往使用 Flash 进行文件预览的方式，在客户端使用 PDF 格式或者 HTML 格式预览。针对此服务在高负载、高并发下的使用场景，本文研究了 PDF 文件按章节切分、存储与传输和基于 MongoDB 冷热数据的均衡。论文的总结如下：

1) 对现有的文件在线预览服务做了调查，由于 Flash 存在占用资源过高、安全性和可靠性低的问题，本文改变了以往依赖 Flash 进行文件预览的方式，而是将多种格式的源文件转换为 PDF 文件格式和 HTML 文件格式在线预览。

2) 由于一些 PDF 文件过大，加上网络带宽的限制，用户对文件某一章节感兴趣时需要传输整个文件，造成用户等待时间过长，影响用户体验。所以本文研究了 PDF 文件解析，将 PDF 文件按章节切分、存储和传输，减少用户等待的时间。

3) 面对高并发的文件转换和切分请求，单个服务器处理受限。本文采用集群的方式来处理请求，通过修改 Nginx 的源代码，将 Nginx 默认的加权轮询算法(weighted round robin)改为节点服务器权重实时变化的动态加权轮询算法(dynamic weighted round robin)。

4) 改进了 MongoDB 的数据均衡策略，对 MongoDB 集群中的热数据均衡问题做了深入研究，提出了基于队列的热数据识别算法和数据块(Chunk)迁移策略，实现了热数据在节点服务器的分布均衡，提高了 MongoDB 集群的性能。

5) 分模块开发了文件在线预览服务的相关工具，降低了系统的耦合性和提高了系统的扩展性，并在高负载网络请求下对文件在线预览服务做了性能测试。

### 下一步工作

本文的下一步工作主要围绕以下几个方面：

1) 分析文件间的相关性，当用户预览某一章节，缓存与此章节相关的文件，减少用户的等待时间。

2) 对 MongoDB 数据均衡的改进中，计算的是数据块的热权重，计算每条数据的热权重，切分数据块，均分数据块的热权重是下一步的工作。

3) 提高文件在线预览服务稳定性和可靠性，增加在各种特殊情况下的处理机制，使得系统更加健壮。

## 参考文献

- [1] 王明月. 企业知识文档检索管理系统的设计与实现[D]. 哈尔滨工业大学, 2017.
- [2] 9月1号起 Chrome 将屏蔽自动播放的 Flash 广告 [Z]. 2015: 2018.//  
<https://36kr.com/p/5036920.html>
- [3] Zomaya A Y, Teh Y. Observations on using genetic algorithms for dynamic load-balancing[J]. IEEE Transactions on Parallel and Distributed Systems. 2001, 12; 12(9; 9): 899-911.
- [4] 曹震寰, 杨莉. Nginx在实现Web加速方面的研究[J]. 甘肃科技. 2015(22): 25-26.
- [5] Liu Y, Wang Y, Jin Y. Research on the improvement of MongoDB Auto-Sharding in cloud environment[Z]. 2012851-854.
- [6] 刘一梦. 基于 MongoDB的云数据管理技术的研究与应用[D]. 北京交通大学, 2012.
- [7] 何杭锋. 基于FODO算法MongoDB自动分片的改进[J]. 计算机技术与发展. 2013(07): 127-130.
- [8] 李东升. 基于Chord环的MongoDB数据均衡系统设计与实现[D]. 重庆大学, 2013.
- [9] 尹君. 基于Markov预测模型的MongoDB分片集群负载均衡策略研究和验证[D]. 云南大学, 2015.
- [10] 王晓琳. MongoDB中的海量数据动态平衡[D]. 上海交通大学, 2014.
- [11] 牛永洁, 薛苏琴. 基于PDFBox抽取学术论文信息的实现[J]. 计算机技术与发展. 2014(12): 61-63.
- [12] 李珍, 田学东. PDF文件信息的抽取与分析[J]. 计算机应用. 2003(12): 145-147.
- [13] 李贵林, 李建中, 杨艳. 用Plug-in实现对PDF文件的信息提取[J]. 计算机应用. 2003(02): 110-112.
- [14] 宋艳娟. 基于XML的HTML和PDF信息抽取技术的研究[D]. 福州大学, 2006.
- [15] 林其水. PDF格式的特点[J]. 今日印刷. 2003(05): 21-25.
- [16] 张秀秀, 张立峰. PDF文件文本内容提取研究[J]. 科技情报开发与经济. 2008(36): 118-120.
- [17] Mcgrath E D K M. A Scalable HTTP Server: The NCSA Prototype[J]. 1994.
- [18] Fang Y, Wang F, Ge J. A Task Scheduling Algorithm Based on Load Balancing in Cloud Computing[C]. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010.
- [19] Dhakal S, Hayat M M, Pezoa J E, et al. Dynamic Load Balancing in Distributed Systems in the Presence of Delays: A Regeneration-Theory Approach[J]. IEEE Transactions on



- Parallel and Distributed Systems. 2007, 18; 18(4; 4): 485-497.
- [20] Moharil S, Lee S. Load Balancing of Parallel Simulated Annealing on a Temporally Heterogeneous Cluster of Workstations[Z]. 2007:1-8.
- [21] 李志成. 基于一致性哈希的高可用负载均衡方案的设计与实现[D]. 浙江大学, 2016.
- [22] 林宏. Nginx反向代理可视化管理方案研究及编程实现[J]. 贵州师范学院学报. 2015(12): 32-35.
- [23] Gu Y, Wang X, Shen S, et al. Analysis of data storage mechanism in NoSQL database MongoDB[Z]. 2015: 70-71.
- [24] Oonhawat B, Nupairoj N. Hotspot management strategy for real-time log data in MongoDB[Z]. 2017: 221-227.
- [25] Wang X, Chen H, Wang Z. Research on Improvement of Dynamic Load Balancing in MongoDB[Z]. 2013: 124-130.
- [26] Chang L, Kuo T. An adaptive striping architecture for flash memory storage systems of embedded systems[Z]. 2002: 187-196.
- [27] Thanalapathi T, Dandamudi S. An efficient adaptive scheduling scheme for distributed memory multicomputers[J]. IEEE Transactions on Parallel and Distributed Systems. 2001, 12; 12(7; 7): 758-768.
- [28] 王晔. 基于HTTP反向代理的Web安全网关技术研究[D]. 扬州大学, 2016.
- [29] 王永辉. 基于Nginx高性能Web服务器性能优化与负载均衡的改进与实现[D]. 电子科技大学, 2015.
- [30] 尚勇. 全国科技工作者数量或超一亿 [Z]. 2017: 2018.// <https://www.antpedia.com/news/74/n-1404674.html>
- [31] Sakr S, Liu A, Batista D M, et al. A Survey of Large Scale Data Management Approaches in Cloud Environments[J]. IEEE Communications Surveys & Tutorials. 2011, 13; 13(3; 3): 311-336.
- [32] Tan Z, Zhou W, Feng D, et al. ALDM: Adaptive Loading Data Migration in Distributed File Systems[J]. IEEE Transactions on Magnetics. 2013, 49; 49(6; 6): 2645-2652.
- [33] 明辉. 缓存那些事[Z]. 2017: 2018.// [https://tech.meituan.com/cache\\_about.html](https://tech.meituan.com/cache_about.html)
- [34] 覃川. 基于Nginx的Web服务器负载均衡策略改进与实现[D]. 西南交通大学, 2017.
- [35] Chang L, Kuo T. An adaptive striping architecture for flash memory storage systems of embedded systems[Z]. 2002: 187-196.
- [36] 闫冠群. 基于数据热点和分片性能差异的MongoDB自适应负载均衡策略研究[D].

浙江师范大学, 2017.

## 攻读硕士学位期间取得的研究成果

一、已发表（包括已接受待发表）的论文，以及已投稿、或已成文打算投稿、或拟成文投稿的论文情况（只填写与学位论文内容相关的部分）：

| 序号 | 作者（全体作者，按顺序排列） | 题 目 | 发表或投稿刊物名称、级别 | 发表的卷期、年月、页码 | 相当于学位论文的哪一部分（章、节） | 被索引收录情况 |
|----|----------------|-----|--------------|-------------|-------------------|---------|
|    |                |     |              |             |                   |         |
|    |                |     |              |             |                   |         |
|    |                |     |              |             |                   |         |
|    |                |     |              |             |                   |         |
|    |                |     |              |             |                   |         |

注：在“发表的卷期、年月、页码”栏：

- 1 如果论文已发表，请填写发表的卷期、年月、页码；
- 2 如果论文已被接受，填写将要发表的卷期、年月；
- 3 以上都不是，请据实填写“已投稿”，“拟投稿”。

不够请另加页。

二、与学位内容相关的其它成果（包括专利、著作、获奖项目等）

1. 《文档在线预览软件 V1.0》 计算机软件著作权 2018 年 7 月

## 致谢

光阴似箭，岁月如梭，近三年的研究生生涯即将结束，回首过去，自己对华工、华工的老师 and 同学充满了感激之情。华工给我创造了良好的学习生活环境；老师不仅教给了自己专业的知识，而且也传授了做人的道理；同学在生活和学习上给予了自己莫大的帮助。在论文即将完成之际，我想向曾经帮助过我的老师和朋友表示最真挚的感谢。

首先，特别感谢我的导师奚建清教授，奚教授是一位治学严谨、和蔼可亲的老师。在我的研究生阶段，奚老师激励自己多多动手实现自己的想法，使想法落地成为可能，这极大的锻炼了自己的动手能力，同时也积累了丰富的开发经验。在论文写作期间，奚老师一丝不苟的严谨态度和耐心细致的指导，让自己少走了很多弯路，顺利完成论文的书写。相信在今后的工作学习中，奚老师对自己的言传身教，会让自己受益终身。

其次，感谢实验室的每一位同学，在学术上我们共同探讨，在生活上我们彼此照顾。在朝夕相处的时间里，让自己的研究生阶段丰富多彩，我相信大家定会成为终身的好朋友。

当然，我还要特别感谢我的家人，无论我身在何地，父母女友都能给予我无私的关怀和帮助，这是自己能够一步步走到今天最为坚实的后盾。

最后，谨向在百忙之中抽空审阅本文的各位专家及评委表示衷心的感谢！

李华青同学的硕士学位论文“文件在线预览服务快速响应技术研究”有较好的实际意义和应用价值。

论文参照了较多的相关文献资料，提出了 1) PDF 文档分片传输机制，2) 提出 Nginx 下动态加权轮询负载均衡优化策略，3) 基于“热数据”的 MongoDB 数据均衡优化策略。并对系统加以实现。通过实验测试，取得了较好的实验效果。

作者对文献的调研、数据的收集较充分，论文资料丰富，论点正确，结论合理，实验数据真实可信。反映作者具有较强的理论基础、较好的科研和工程能力。论文结构清晰，图文表达规范，达到硕士学位论文水平。

学位申请人答辩过程讲述清楚，对评阅意见中提出的问题和质疑已作出明确的回复，答辩委员判定学位申请人的回复已达到评阅专家的要求。经答辩委员会无记名投票，同意该同学通过硕士学位论文答辩，同意授予硕士学位。

论文答辩日期：2019 年 03 月 14 日

答辩委员会委员 5 人

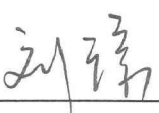
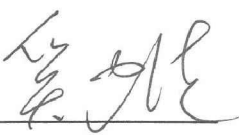
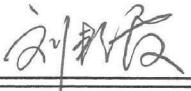
表决票数：同意毕业及授予学位（5）票；

同意毕业，但不同意授予学位（0）票；

不同意毕业（0）票

表决结果（打“√”）：通过（√）； 不通过（ ）

决议：同意授予硕士学位（√） 不同意授予硕士学位（ ）

|                           |                                                                                          |                                                                                     |                                                                                       |
|---------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 答辩<br>委员<br>会成<br>员签<br>名 |  (主席) |  |  |
|                           |       |  |  |